

XBRL GL Next Taxonomy Framework

— Part 1: General rules

Document status: Draft — For Collaborative Review

Author: Nobuyuki SAMBUICHI

Role: Advisor to XBRL Japan

Project: XBRL GL Next

Distribution: Private project repository — invited participants only

This document is a draft under development by Nobuyuki SAMBUICHI, Advisor to XBRL Japan, for collaborative review within the XBRL GL Next project. It is shared through a private GitHub repository with invited project participants. It has not been approved or endorsed as an official specification by XBRL Japan, XBRL Europe, XBRL International, or any other organization.

Copyright and license terms under review.

Revision 2026-07-29: Clause 3, Terms and definitions, is mandatory. This revision defines the semantic-model layers, root-specific BSM and HMD distinction, processing terms, module and Profile concepts, Palette model, entry points, and extension/build records. The original Specialisation and Graph Walk processing flows are unchanged.

Revision 2026-07-29 (FSM_q alignment): The normative FSM column contract is the 15-column structure in FSM_q.xlsx. `association\_role` is a separate column immediately after `property\_term`. The BSM adds only `id`, producing 16 columns, and the LHM remains 17 columns. Association identity is (`association\_role`, `associated\_module`, `associated\_class`); neither `property\_term` nor a Name or QName column participates in that identity. For a Reference Association whose target has no `identifier=PK` Attribute, Graph Walk records a non-fatal error, emits no row below the R occurrence, and continues with the originating Class.

Current design baseline for collaborative review

Revision 2026-07-28: This revision makes the FSM the authoritative declaration of modules, Classes, properties, roots, and constraints; removes any requirement for separate profile data; specifies specialisation and Graph Walk in detail, including LIFO processing frames and associated-Class traversal; clarifies modules, Profiles, and the Palette model; and distinguishes the extension-result JSON from the build-manifest generation record.

The following statements define the current PoC baseline used when interpreting this draft. Earlier text that describes an official publication, a final contract, or an unregistered artefact is superseded by this baseline for the collaborative review.

- **Semantic model:** The semantic model consists only of the Foundational Semantic Model (FSM), Business Semantic Model (BSM), and Logical Hierarchical Model (LHM). In the current PoC exchange contract, FSM, BSM, and LHM use 15, 16, and 17 CSV columns respectively. The FSM and BSM hold `property\_term` and `association\_role` in separate columns. The BSM does not contain an `element` column; Graph Walk derives `element` only after the `semantic\_path` is known.
- **BSM identity:** Each FSM module is assigned an FSM-wide unique two-letter uppercase alphabetic abbreviation XX. A Class is assigned XXnn, where nn is its order of occurrence in that module. Its properties are assigned XXnn-01, XXnn-02, and so on, in order of occurrence within the Class. If a Class has 100 or more properties, all property sequence numbers for that Class use the number of digits required by the maximum property number, for example XXnn-001

through XXnn-100. IDs shall be unique across the integrated semantic model. The BSM id is carried unchanged into the LHM; repeated LHM occurrences are distinguished by semantic_path.

- LHM and HMD: The LHM is the final semantic-model layer and represents the logical hierarchy generated from the BSM. An HMD is a root-specific projection used for syntax binding; it is not a fourth semantic-model layer and is not an alias for the LHM.
- **Syntax binding and HMD:** A Hierarchical Model Definition (HMD) is introduced only for syntax binding. It is a document structure derived from a specified root Class in the LHM and its reachable properties; it is not a semantic-model layer or an alias for the LHM. For example, a Sales Invoice HMD is derived from the Sales Invoice Class in the LHM. XSD, linkbases, JSON metadata, and CSV layouts are syntax-binding artefacts. The proposed semantic-binding and syntax-binding sheets are provisional proposals subject to PoC implementation and collaborative review, and are not final contracts.
- **Shared, Aligned, and Distinct:** Shared definitions are suitable for broad cross-domain reuse. Aligned definitions preserve alignment with a regional or national standard, legislation, regulation, or a published industry standard, including a specific business context, process, or document structure. Distinct definitions are specific to an enterprise, corporate group, product, bilateral trading relationship, or closed community. Origin in CCL or another international standard does not by itself make a concept Shared.
- Property and Association processing: An Attribute is identified by `property\_term`. An Association is identified by the ordered combination of `association\_role`, `associated\_module`, and `associated\_class`; `property\_term` is not part of Association identity. Association kind and multiplicity are characteristics that a child Class may change. Multiplicity 0 or 0..0 deletes an inherited property. Associations from a concrete Class to an Abstract Class are reported and excluded from the BSM.
- **Class and namespace resolution:** Logical Class references use explicit module and Class information. QName processing belongs to syntax binding, where a QName is separated into prefix and local name. A namespace is never inferred from the first three characters of a Class name. Unknown prefixes, undefined Classes, and ambiguous unqualified Class names are input errors.
- **Artefact status:** The taxonomy dated 2026-12-31 is a technical validation baseline, not an official release. A separate profile data file is not required by the current processing model. Where the term "profile" is used, it denotes an identifiable and versioned use-case configuration derived from an FSM, one or more root Classes, and the applicable syntax bindings.

Repository version: private collaborative review draft.

Editors: SAMBUICHI, Nobuyuki

Contributors: invited project participants (individual names not yet confirmed for publication).

Status

This document is a draft under development by Nobuyuki SAMBUICHI, Advisor to XBRL Japan, for collaborative review within the XBRL GL Next project. It is shared through a private GitHub repository with invited project participants. It has not been approved or endorsed as an official specification by XBRL Japan, XBRL Europe, XBRL International, or any other organization.

Abstract

This document defines the general technical and modelling rules for the XBRL GL Next taxonomy framework. It specifies:

- a layered semantic model structure — FSM (Foundational Semantic Model), BSM (Business Semantic Model), and LHM (Logical Hierarchical Model) — and deterministic Graph Walk processing from BSM to the domain-wide LHM;
- modularisation (core, business, measurable items, document, accounting book, multi-currency, tax, cross-cutting capability, and jurisdiction modules) and profiles to satisfy the functional requirements in the XBRL GL Next Requirements Specification;
- extension mechanisms and governance using Shared, Aligned, and Distinct extensions, including an aligned pool (alignment set) registry for cross-jurisdiction reuse.

This Part 1 is serialisation-independent. XBRL GL Next Taxonomy Framework — Part 2: XBRL 2.1 palette taxonomy and XBRL GL Next Taxonomy Framework — Part 3: xBRL-CSV palette taxonomy specify the serialisation-specific rules for XBRL XML (Tuple) and XBRL OIM (xBRL-CSV) respectively.

Table of Contents

Current design baseline for collaborative review	1
Status	2
Abstract.....	3
1 Language Independence.....	7
2 Introduction.....	7
2.1 Scope of this document.....	7
2.2 Normative language and document conventions	7
2.3 Relationship to Parts 2, 3, and 4.....	7
3 Terms and definitions	7
3.1 General	8
3.2 Semantic-model layers	8
3.3 Semantic processing.....	9
3.4 Modules, Profiles, and taxonomy assembly.....	10
3.5 Extension and build records.....	10
4 Requirements and Motivation.....	11
5 Framework Architecture Overview.....	13
5.1 Semantic-model layers and processing pipeline	13
5.2 Processing pipeline.....	14
5.2.1 Specialisation (FSM → BSM)	14
5.2.2 Graph Walk (BSM → LHM/HMD)	14
5.2.3 Reproducibility.....	15
5.3 Modules and taxonomy assemblies	15
5.3.1 Module types	15
5.3.2 Use-case taxonomy assemblies.....	15
5.3.3 Semantic and syntax bindings.....	15
5.4 Naming Convention.....	15
5.4.1 General requirement.....	15
5.4.2 Identifier Structure	16
5.4.3 Generation and Normalization Process.....	16
5.4.4 Specialization of Reusable Classes	17
5.4.5 Inheritance and Module Ownership	17
5.4.6 Conformance and Validation	17
5.4.7 Rationale.....	17
5.5 Semantic Path	17
5.5.1 Purpose and Reference.....	17
5.5.2 Syntax Rules and Construction	17
5.5.3 Functional Role in the Framework.....	18
5.5.4 Conformance and Validation	18
6 Extension Mechanisms and Governance	18
6.1 Semantic authority and extension point.....	18
6.2 Shared, Aligned, and Distinct	18
6.3 Namespace, ownership, and versioning.....	19
6.4 Extension property identity	19
6.5 Extension result manifest.....	19
6.6 Aligned-pool governance.....	20
6.7 Build manifest and its boundary.....	20
7 Graph walk	20
7.1 Scope and processing stages.....	20

7.2 Specialisation: FSM to BSM.....	21
7.2.1 Inputs and registration.....	21
7.2.2 QName and Class resolution.....	21
7.2.3 Inheritance resolution.....	21
7.2.4 Property identity.....	23
7.2.5 Override, addition, and deletion.....	23
7.2.6 Abstract Classes and invalid Associations.....	23
7.2.7 BSM output contract.....	23
7.2.8 Specialisation conformance.....	24
7.3 Graph Walk: BSM to LHM.....	24
7.3.1 Inputs and root selection.....	24
7.3.2 Class map and property order.....	24
7.3.3 Depth-first traversal.....	24
7.3.4 Association semantics.....	26
7.3.5 Cycles and repeated occurrences.....	27
7.3.6 Semantic path.....	27
7.3.7 Element-name derivation.....	27
7.3.8 LHM output contract.....	27
7.3.9 Graph Walk conformance.....	27
7.4 HMD derivation.....	27
7.5 Current PoC command mapping (informative).....	28
7.6 Minimum test set.....	28
8 Modules, Profiles, and the Palette Model.....	28
8.1 Module concept.....	28
8.2 Baseline module composition.....	28
8.3 Module dependencies and ownership.....	29
8.4 Profile.....	29
8.5 Profile realisation.....	29
8.6 Palette model.....	29
9 Palette Taxonomy.....	30
9.1 Purpose.....	30
9.2 XBRL 2.1 palette taxonomy.....	30
9.3 OIM-oriented taxonomy.....	30
9.4 Message entry points and DTS identity.....	30
9.5 Extension records.....	30
10 How to create extensions within the XBRL GL Next Taxonomy Framework.....	30
10.1 Define the semantic extension.....	30
10.2 Validate and specialise.....	31
10.3 Generate hierarchies.....	31
10.4 Generate taxonomy bindings.....	31
10.5 Record extension results.....	31
10.6 Validate and record the build.....	31
11 Syntax Binding.....	31
11.1 Binding source.....	31
11.2 Binding information.....	31
11.3 XBRL 2.1 and OIM equivalence.....	31
11.4 Validation.....	32
12 Reverse engineering and migration support (informative).....	32
12.1 Purpose.....	32
12.2 Background: XBRL GL XML (XBRL 2.1 / XML realisation) and implicit semantics.....	32
12.3 Principle of semantic primacy.....	32

12.4 Role of the Foundational Semantic Model (FSM)	32
12.5 Reconstruction of semantic layers	33
12.6 Regeneration and dual syntax support.....	33
12.7 Summary	33
Appendix A (illustrative) Example: Extending “Sales Order” and “Purchase Order” from abstract class “Order”	34
A.1 Two-phase architecture	34
A.1.1 Specialization Processing (Phase 1 of Graph Walk)	34
A.1.2 Normal Graph Walk Processing (Phase 2 of Graph Walk)	34
A.2 Abstract class 'Order' and its specialization.....	36
Appendix B (illustrative) Example of the normal navigation graph walk.....	39
B.1 Graph Walk: Defining the LHM by copying the BSM	39
B.2 The graph walk process for the root class A.....	40
Appendix C (illustrative) Semantic Layers of the xBRL GL Framework.....	43
C.1 Foundational Semantic Model (FSM)	43
C.2 FSM Extension.....	43
C.3 Business Semantic Model (BSM)	43
C.4 Logical Hierarchical Model (LHM)	43
C.5 HMD and HMD root.....	44
C.6 Profile and message entry points	44
C.7 Palette model	44
C.8 Extension and build records.....	44
Appendix D (illustrative) Semantic Layers Example: defining an extension in the FSM and generating the resulting BSM and LHM	45
D.1 Purpose and scope	45
D.2 Inputs (example sheets)	45
D.3 Design policy used in the example	45
D.4 Step-by-step procedure (FSM → BSM → LHM).....	45
D.4.1 Step 1 — Define the extension in the FSM (FSM_btx extends FSM)	45
D.4.2 Step 2 — Specialise to create the BSM (select necessary elements).....	46
D.4.3 Step 3 — Generate the LHM via deterministic graph walk	46
D.5 Conformance checks recommended for this workflow.....	46
D.6 Expected outputs and how they are used	47
Appendix E (illustrative) Example: “VAT category taxable amount” (EN 16931 BT-116)	48
E.1 UBL 2.1 (Peppol BIS Billing 3.0 – UBL Invoice)	48
E.2 UN/CEFACT CII D16B (EN 16931 CII)	48
E.3 Example syntax binding table entry	48
E.4 Syntax binding example: XBRL GL tuple palette taxonomy	49
Appendix F References	51
Appendix G (illustrative) Intellectual property status.....	52
Appendix H (illustrative) Document history.....	53
Appendix I (illustrative) Errata corrections in this document	54

1 Language Independence

This collaborative review draft is written in English, and the preferred spelling convention is UK English.

2 Introduction

2.1 Scope of this document

This document defines the general technical and modelling rules for the XBRL GL taxonomy framework. Part 1 of the specification is serialisation-independent. The overall framework is designed to satisfy functional requirements, including the need for a single shared semantic model to be represented consistently across both legacy XBRL 2.1 tuple-based syntax and state-of-the-art XBRL Open Information Model (OIM)-based serialisations.

This document defines general rules for the organisation and modelling of XBRL GL taxonomy artefacts and their extension mechanisms. It specifies design rules for:

- layered semantic modelling and deterministic hierarchy generation;
- modular packaging, palette/profile-based assembly, and jurisdiction profiling; and
- aligned cross-jurisdiction extension governance (including the aligned extension registry and aligned pool).

This document does not define jurisdiction-specific filing rules, authority-specific submission interfaces, or operational exchange procedures. Such requirements are expected to be expressed via jurisdiction modules and profiles.

2.2 Normative language and document conventions

The key words "MUST", "MUST NOT", "REQUIRED", "MUST", "MUST NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in RFC 2119.

Illustrative examples and counterexamples (if provided) are explicitly marked. Italics are used for rhetorical emphasis only and do not convey any special conformance meaning.

2.3 Relationship to Parts 2, 3, and 4

This Part 1 defines serialisation-independent rules and the common semantic processing contract used by the companion parts. Part 2 defines the XBRL 2.1 palette-taxonomy binding, Part 3 defines the OIM/xBRL-CSV binding, and Part 4 defines governance of the Aligned pool and reusable extensions.

- XBRL GL Next Taxonomy Framework — Part 1: General rules;
- XBRL GL Next Taxonomy Framework — Part 2: XBRL 2.1 palette taxonomy;
- XBRL GL Next Taxonomy Framework — Part 3: xBRL-CSV palette taxonomy.
- XBRL GL Next Taxonomy Framework — Part 4: Aligned pool for extension.

3 Terms and definitions

For the purposes of this document, the following terms and definitions apply. Where a term is used in a processing flow, script, test, JSON record, taxonomy, or related Framework part, it MUST have the meaning specified in this Clause.

Terminology precedence. If an implementation label or legacy artifact uses a different label, this Clause governs conformance unless an explicit compatibility mapping is documented. In particular, a root-specific BSM is not an HMD until Graph Walk has established the hierarchy and semantic paths.

3.1 General

normative

describes a provision with which conformance is required. The key words defined in 2.2 indicate the degree of obligation.

informative

describes explanatory material, examples, implementation notes, or other material that does not establish a conformance requirement.

QName

a namespace-qualified name consisting of a namespace URI, represented by a registered prefix in an input file, and a local name. A QName MUST be resolved by the registered prefix and local name; a namespace MUST NOT be inferred from the first characters of a Class name.

Class

a semantic-model construct that defines a type of business object and owns or inherits properties.

property

an Attribute or Association declared for, or inherited by, a Class.

Attribute

a property that represents a value and its datatype rather than a relationship to another Class.

Association

a property that relates an originating Class to an associated Class.

association role

the name, held separately from property term, that identifies the role of an associated Class in relation to its originating Class. It distinguishes multiple Associations to the same associated Class.

associated Class

the target Class identified by the ordered pair of ``associated_module`` and ``associated_class`` in an Association. Specialisation validates this logical Class reference; Graph Walk follows it in accordance with the Association kind.

multiplicity

the minimum and maximum permitted occurrences of a property. An effective multiplicity of ``0`` or ``0..0`` is a deletion directive and MUST NOT be emitted as an effective BSM, LHM, HMD, or taxonomy property.

3.2 Semantic-model layers

Foundational Semantic Model (FSM)

the authoritative declarative semantic model containing modules, Classes, properties, Specialization relationships, candidate root Classes, constraints, and their provenance before inheritance is resolved.

Business Semantic Model (BSM)

the effective semantic model produced by Specialisation after inheritance, additions, overrides,

deletions, QName references, and applicable validation rules have been resolved. A BSM is not yet a hierarchical traversal output and does not contain final semantic paths.

root-specific BSM

a BSM produced when Specialisation is invoked with an explicit root Class. It contains the effective Classes and properties selected as the source for one HMD. It is an intermediate BSM and **MUST NOT** itself be designated as an HMD.

Logical Hierarchical Model (LHM)

the hierarchical model produced by Graph Walk from a module-wide BSM that was generated without an explicit root-Class selection in Specialisation. It records the applicable occurrence hierarchy, order, multiplicities, semantic paths, and element names for the module-wide logical view.

Hierarchical Message Definition (HMD)

the root-specific hierarchical model produced by Graph Walk from a root-specific BSM. An HMD defines the occurrence hierarchy, order, multiplicities, semantic paths, and element names for one message, ledger, report, or other root-bounded information set.

HMD root

the resolved root Class QName explicitly supplied to Specialisation and used as the traversal origin for the corresponding HMD. Journal Entry, Trial Balance, Order, and Invoice are examples of possible business meanings; identity is determined by the root Class QName.

3.3 Semantic processing

Specialisation

the deterministic FSM-to-BSM process that resolves superclass inheritance, property identity, additions, overrides, deletion directives, QName references, and invalid Associations. When a root Class is supplied, it produces a root-specific BSM.

Graph Walk

the deterministic depth-first process that traverses the BSM from the applicable origin, follows associated Classes according to Association kind, and produces an LHM or HMD with semantic paths and element names.

LIFO list

the last-in, first-out list used by the original normative processing flow to record the current Class traversal path and support return to the originating Class after the associated Class has been processed. The prescribed PUSH and POP order is part of the processing flow.

active resolution stack

the Specialisation LIFO structure containing the chain of Classes whose inheritance is currently being resolved. It is distinct from a cache of already resolved Classes.

active traversal path

the sequence of resolved Class QNames currently held for Graph Walk. It is used for cycle detection and **MUST** be distinguished from an occurrence name containing an Association role.

occurrence

a context-specific appearance of a Class or property in an LHM or HMD. The same resolved Class **MAY** occur on different non-cyclic paths under different Association roles.

semantic path

a stable, human-readable identifier derived by Graph Walk from the ordered occurrence path. It identifies semantic context independently of a particular XML, JSON, or CSV serialisation.

Composition

an Association kind for which Graph Walk emits the associated Class and traverses its effective properties as part of the containing hierarchy.

Aggregation

an Association kind for which Graph Walk emits the associated Class and traverses its effective properties while retaining the aggregation semantics prescribed by the model.

Reference Association

an Association kind for which Graph Walk emits the reference occurrence and, when present, the target Class's `identifier=PK` Attribute or Attributes as `REF`, without traversing target non-key Attributes or Associations. If the target has no `identifier=PK` Attribute, Graph Walk emits no row below the reference occurrence, records a non-fatal error diagnostic, and continues with the next property of the originating Class.

property identity

the key used to match inherited and child properties: `property\_term` for an Attribute, and `association\_role`, `associated\_module`, and `associated\_class` for an Association. `property\_term`, Association kind, representation term, and multiplicity do not form part of Association identity.

3.4 Modules, Profiles, and taxonomy assembly

module

a versioned, namespace-owned unit of reusable semantic definitions and the corresponding taxonomy components. A module boundary expresses ownership and dependency and is not merely a directory boundary.

Shared

the reuse category for concepts and structures whose meaning is valid across countries, regimes, industries, and implementations.

Aligned

the reuse category that specialises or constrains Shared definitions to maintain alignment with a stated international, regional, national, legal, regulatory, or public industry context.

Distinct

the extension category for definitions specific to an enterprise, enterprise group, product, bilateral relationship, or closed community.

Profile

an identifiable use-case configuration that selects one or more modules, HMDs, message entry points, constraints, and permitted serialisations. A Profile is not an independent semantic-model layer and does not replace the FSM, BSM, LHM, or HMD.

Palette model

the taxonomy implementation approach in which reusable module taxonomies are selected and assembled for a Profile or message entry point without duplicating their authoritative semantic definitions.

message entry point

the physical XBRL taxonomy entry point that realises one HMD and loads the module taxonomy components required for its HMD root. The basic relationship is one HMD root to one HMD to one message entry point.

syntax binding

the rules and artifacts that map a semantic model or hierarchical model to an XBRL 2.1, xBRL-CSV, xBRL-JSON, or other physical serialisation.

3.5 Extension and build records

extension manifest

the JSON record of the semantic result of an Aligned or Distinct extension, including provenance, additions, specialisations, constraints, deletions, and referenced code lists. It records the extension result and is not a build log.

build manifest

the JSON generation record containing input identities and hashes, processor and tool versions, commands, output hashes, and validation results. It supports reproducibility but MUST NOT define or override semantic content.

taxonomy entry point

an XBRL schema or other DTS entry artifact through which a taxonomy assembly is discovered. In this framework, a message entry point is the root-specific taxonomy entry point corresponding to one HMD.

4 Requirements and Motivation

The XBRL GL taxonomy framework is designed to satisfy functional requirements including:

- a) business documents (order/purchase, shipment/logistics, invoicing, payments) with at least a three-layer structure: document header, tax category totals, line items;
- b) accounting books (daily journal/journal entries, trial balance, general ledger, AP/AR ledgers) including adjustments, cash application, and cash payment transactions;
- c) monetary amounts and (sub)totals at appropriate levels, including line totals, taxable/basis amounts, tax amounts by category and/or type, paid amounts, and payable/outstanding amounts;
- d) statistical and other non-financial records captured at transaction, document, and/or measurable-item level, including (as applicable) work time and attendance, inventory movement and quantities, production or service metrics, payment statistics, logistics/performance measures, and environmental/ESG measures; and
- e) cross-cutting capabilities: multi-currency, business segment and accounting segment classification, system user activity logging (audit trail), and drill-down/roll-up to/from annual report structures (e.g., BS/P&L).

In addition, the framework is designed so that a single shared semantic model can be represented consistently across multiple syntaxes, including legacy XBRL 2.1 tuple-based serialisations and state-of-the-art OIM-based serialisations. To support extensibility in shared, aligned, and distinct forms, the semantic model is organised into three layers (FSM: Foundational Semantic Model, BSM: Business Semantic Model, and LHM: Logical Hierarchical Model).

The framework also provides an extension mechanism through the palette taxonomy framework, enabling jurisdictions to publish and reuse extensions in a consistent manner. Extensions intended for cross-jurisdiction reuse are supported through an aligned extension sharing pool (aligned pool) of semantic model components, so that comparability can be maintained while allowing jurisdiction-specific requirements.

These proposed requirements are defined in the companion “XBRL GL Next Requirements Specification” and are referenced here only to motivate the framework architecture and module boundaries.

The following table maps the requirements from the Requirements Specification to their support and implementation in this part of the framework.

Table 1 — Mapping of XBRL GL Next Requirements

Requirement ID	Requirement Name	Supported in Part 1 (General Rules)
GL-REQ-0001	Common use cases	The framework is explicitly designed to satisfy requirements for business documents, accounting books, and tax reporting.
GL-REQ-0002	Bridge from transactional standards to reporting	The Layered Semantic Model (FSM, BSM, LHM) is designed to provide a single shared model bridging transactions to serialisations.

Requirement ID	Requirement Name	Supported in Part 1 (General Rules)
GL-REQ-0003	General ledger and subsidiary ledgers	The framework mandates module support for accounting books, including journals, ledgers (GL/AP/AR), and trial balances.
GL-REQ-0006	Non-financial extensions	The framework supports statistical and non-financial records (e.g., ESG, work time) via specific modules.
GL-REQ-0101	Core structure centred on journal entry	The framework requires support for business documents (header/lines), accounting books, multi-currency, and audit trails.
GL-REQ-0104	Amounts, currencies, quantities...	The framework requires monetary amounts, subtotals, and tax amounts at appropriate levels.
GL-REQ-0105	Tax modelling	The framework supports semantic binding to tax category/type schemes and totals.
GL-REQ-0201	Modular taxonomy architecture	The framework "MUST be modular," defining core, business, jurisdiction, and cross-cutting modules.
GL-REQ-0202	Cross-jurisdiction alignment	Extensions intended for reuse must be managed as "Aligned extensions" via an aligned pool to maintain comparability.
GL-REQ-0204	Localisation ownership & governance	Jurisdiction modules must be separate from international modules and registered in the aligned extension registry.
GL-REQ-0205	Backward compatibility	Extensions should maintain backward compatibility, or provide versioning/migration provisions.
GL-REQ-0301	Support for XML and OIM	The framework is explicitly designed for both XBRL 2.1 (tuple-based) and OIM serialisations.
GL-REQ-0303	Deterministic transformability	The "Graph Walk" process ensures a deterministic hierarchy is generated from the BSM, enabling consistent transformation.
GL-REQ-0402	Clear profile definitions	Defines a "Profile" as a declared set of constraints, modules, and serialisation rules.

5 Framework Architecture Overview

5.1 Semantic-model layers and processing pipeline

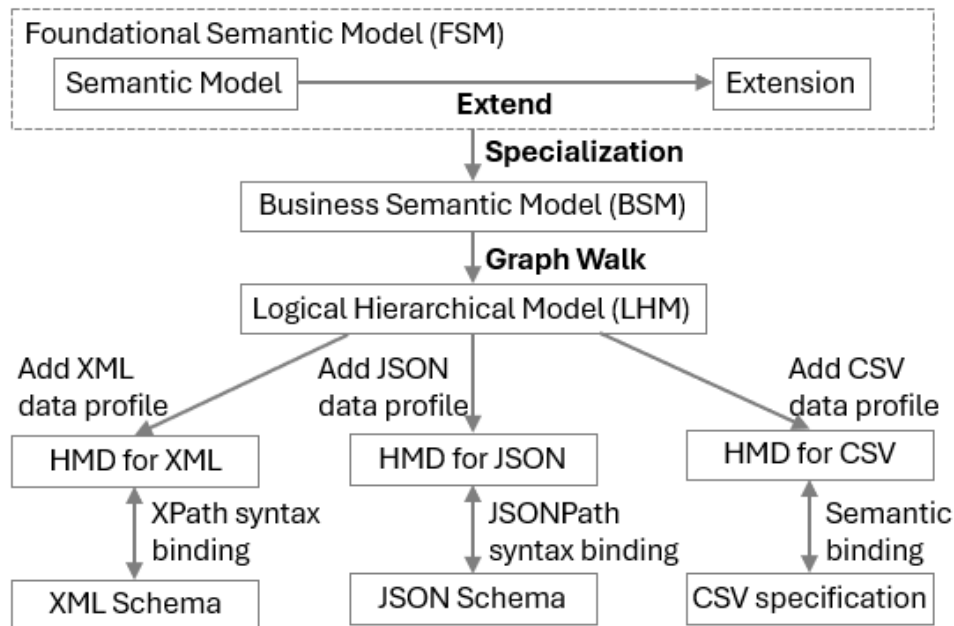


Figure 1 — Layered Semantic Model

The framework MUST be organised into three logical semantic layers, as shown in Figure 1.

a) Foundational Semantic Model (FSM)

The FSM defines jurisdiction-neutral classes, properties, attributes, and associations that represent reusable business and accounting semantics and controlled extension points. The FSM also includes an aligned set (aligned pool) of classes and properties that MAY be selected in a BSM to meet jurisdictional requirements. A jurisdiction BSM is derived by selecting classes from the FSM and/or specialising FSM super classes, selecting applicable classes and properties, and MAY add jurisdiction-specific properties where necessary (as aligned or distinct extensions, as applicable).

b) Business Semantic Model (BSM)

A BSM refines the FSM to meet a bounded scope (jurisdiction and/or domain) by:

- selecting necessary classes and properties;
- constraining multiplicities and value domains;
- defining specialisations (subclasses) to represent jurisdictional requirements.

c) Logical Hierarchical Model (LHM)

The LHM is a hierarchical representation generated from the BSM by the graph-walk process defined in 4.2. The LHM provides stable hierarchical structures used as the basis for serialisation bindings (Part 2 and Part 3).

The framework **MUST** be organised into three, and only three, semantic-model layers. Syntax artefacts **MUST NOT** be treated as additional semantic layers.

Artefact	Role	Authority and derivation
FSM	Declares modules, Classes, properties, inheritance, Associations, roots, multiplicities, value domains, and extensions.	Authoritative semantic source edited by model authors.
BSM	Contains the effective concrete Classes and properties after inheritance, overrides, additions, and deletions have been resolved.	Generated deterministically from one or more FSM CSV sheets by specialisation.
LHM	Represents the logical hierarchy produced by walking the resolved BSM from declared roots.	Generated deterministically from the BSM.
HMD	Provides a root-specific hierarchy for a document, ledger, transaction, or report binding.	Derived from the LHM for syntax binding; not a semantic model layer.
Taxonomy	Implements an HMD and its semantic identities in XBRL 2.1 or OIM-oriented XBRL constructs.	Generated syntax-binding artefact.
Profile	Identifies a use-case configuration: module set, root Class or Classes, HMDs, applicable constraints, and permitted serialisations.	Configuration concept realised by entry points; not a semantic-model layer or duplicate semantic dataset.
Palette model	Provides reusable module taxonomies and rules for assembling them into a Profile-specific DTS.	XBRL implementation model; it does not author semantic content independently of the FSM.

The FSM is the source of truth for the selected module set, Class set, property set, root Classes, and semantic constraints. The processing model defined by this Part does not require `profile.json`, a separate selection sheet, or another profile dataset that duplicates the FSM.

A Profile is an identifiable and versioned use-case configuration, such as Accounting Entries or Japanese Accounting Ledger. It states which semantic modules and roots form the intended scope, which HMDs and constraints apply, and which serialisations are supported. These choices **MUST** already be resolvable from the FSM and build inputs; the Profile **MUST** not redefine Classes or properties. Its physical XBRL realisation is the relevant taxonomy entry point and DTS.

5.2 Processing pipeline

The normative processing sequence is FSM → BSM → LHM/HMD → taxonomy. Specialisation and Graph Walk **MUST** operate without requiring profile data.

5.2.1 Specialisation (FSM → BSM)

Specialisation resolves Class inheritance and property directives. It copies inherited properties, applies child overrides and additions, removes properties whose effective multiplicity is `0` or `0..0`, excludes abstract Classes from the concrete BSM output, and reports invalid references. The detailed requirements are specified in Clause 7.2.

5.2.2 Graph Walk (BSM → LHM/HMD)

Graph Walk starts from one or more explicitly supplied root Class QNames, traverses the resolved Associations in a stable order, emits Class and Attribute occurrences, represents Reference Associations by identifiers, and derives stable semantic paths and element names. The detailed requirements are specified in Clauses 7.3 and 7.4.

5.2.3 Reproducibility

Given byte-identical FSM inputs, the same root Class sequence, the same processing rules, and the same tool version, the BSM, LHM/HMD, and generated taxonomy MUST be reproducible. A build manifest MAY record those conditions as specified in Clause 6.7, but it MUST NOT redefine semantic content.

5.3 Modules and taxonomy assemblies

5.3.1 Module types

The framework MUST support modular taxonomy packaging, including at least:

- a) Core module: jurisdiction-neutral foundational concepts required for minimum interoperability;
- b) Business document modules: order/purchase, shipment/logistics, invoicing, payments/settlement;
- c) Accounting book modules: journal entries, ledger structures (GL/AP/AR), trial balance, and adjustments including cash application/payment;
- d) Cross-cutting capability modules: multi-currency, segment classification, activity logging, reporting drill-down/roll-up mappings, statistical/non-financial measures;
- e) Jurisdiction modules: localisation (labels, code lists, constraints, and mandated sets).

5.3.2 Use-case taxonomy assemblies

A use-case taxonomy assembly is the named combination of semantic modules and root-specific structures implemented by a taxonomy entry point. The FSM already declares the Classes, properties, roots, and constraints used to generate that assembly.

An implementation MAY publish human-readable assembly metadata or a machine-readable build configuration, but such metadata MUST reference rather than duplicate the FSM. Conflicts MUST be resolved in favour of the FSM and the generated BSM/LHM traceability chain.

A Profile therefore has four responsibilities: identify the intended business use; identify the module versions and root Class or Classes used to derive HMDs; identify applicable validation and code-list constraints; and identify the serialisation-specific entry points. It has no responsibility for defining semantic Classes, properties, inheritance, or Association identity.

5.3.3 Semantic and syntax bindings

The framework MUST support:

- a) semantic binding to controlled vocabularies and code lists (including tax category/type schemes and other classifiers);
- b) syntax binding to the serialisations defined in Part 2 and Part 3.

Where an implementation supports both XBRL XML and OIM serialisations, the framework SHOULD enable deterministic transformation without semantic loss within the bounds of a declared profile.

5.4 Naming Convention

5.4.1 General requirement

XBRL linkbases require unambiguous identification of taxonomy components to locate concept definitions within the taxonomy schema. In the XBRL binding, these components are identified using a unique id attribute.

To satisfy these requirements, each class and attribute represented in the Logical Hierarchical Model (LHM) MUST have a unique identifier suitable for XBRL taxonomy and linkbase generation. Each identifier MUST be constructed using the pattern <module> "_" <elementName> to ensure the combination is globally unique across the entire taxonomy set.

These semantic identifiers MUST be used consistently across the following layers of the framework:

- Foundational Semantic Model (FSM)
- Business Semantic Model (BSM)
- Logical Hierarchical Model (LHM)
- Generated tuple-based and OIM-based taxonomies

This consistency ensures that the structural and semantic meaning of a concept remains traceable from its high-level definition down to its physical serialization.

5.4.2 Identifier Structure

Each semantic identifier MUST follow a standardized pattern:

`<module> ":" <normalizedElementName>`.

- `<module>`: The stable 3-letter module identifier (e.g., `cor`, `bus`, `btx`, `sta`).
- `<normalizedElementName>`: A unique name derived from class or property terms, formatted using Lower Camel Case Concatenation (LC3).

5.4.3 Generation and Normalization Process

Element names MUST be generated through a deterministic process that transforms terms from the BSM into normalized XML names for the LHM.

5.4.3.1 Hierarchical Path Integration

The position of an element within the LHM governs its name generation:

- Level 1 Elements: Names for root classes are derived directly from the primary class term.
- Level 2 and Above: Names for properties and nested classes MUST incorporate structural context by evaluating the semantic path from the root to the current node.
- Module Namespace: Every element name MUST be prefixed with its respective module identifier followed by a colon (e.g., `btx:`).

5.4.3.2 Normalization and Deduplication

To ensure concise taxonomies, the following rules MUST be applied to concatenated terms:

- Word Extraction: Component terms (Class, Property, and Representation) are split into individual words based on spaces or CamelCase boundaries.
- Sequence-Preserved Deduplication: When merging a parent class name with a property name, any duplicate words MUST be removed while maintaining word order.
- Case Neutrality: Deduplication is performed case-insensitively, with the final output converted to LC3.

Example: If a property `DocumentDate` is located under class `TransactionHeader`, the resulting name is `TransactionHeaderDocumentDate`. If the property were named `HeaderDocumentDate`, the duplicate word "Header" is removed to produce the same normalized result.

5.4.3.3 Handling Specialization and Inheritance

Naming logic ensures traceability during the FSM to BSM transition:

- Inheritance Resolution: Properties inherited from abstract superclasses MUST be copied to the subclass before naming.
- Property Overrides: If a subclass specializes a property (e.g., changing multiplicity), the new BSM definition drives the element name generation.
- Contextual Suffixes: For reference associations, the name MUST be formatted as `{associationRole}_{associatedClass}` to distinguish the relationship from the class itself.

5.4.4 Specialization of Reusable Classes

Reusable semantic classes defined in the FSM MAY be specialized for different structural scopes (e.g., Header vs. Detail).

- Semantic Inheritance: Specialized classes inherit the semantics of the reusable base class (e.g., `Amount`).
- Distinct Identification: These classes receive unique element names based on their structural position in the LHM.

Example: A reusable `Amount` class specialized for different scopes:

``btx:headerAmountStructure` `btx:detailAmountStructure``

- Attribute Consistency: Attributes within these classes follow the same scope-based naming (e.g., `btx:headerAmount` and `btx:detailAmount`).

5.4.5 Inheritance and Module Ownership

Rules for attributes inherited from abstract classes (e.g., from `Header` or `Detail`):

- Element Stability: The element name MUST remain unchanged during inheritance.
- Namespace Identity: The module name identifies the namespace where the element is defined.
- Semantic Reuse: Inherited attributes SHOULD retain the module of their original definition to promote reuse.
- Divergence: If intentional semantic divergence is required, a new element name MUST be defined.

5.4.6 Conformance and Validation

Conformance MUST be verified in both BSM and LHM representations. At minimum, the following machine-readable checks MUST be satisfied:

- All `id` values are unique.
- All `element\_id` values are unique.
- No two attributes differ only by structural placement without a corresponding scope prefix or contextual change in their normalized names.

5.4.7 Rationale

This convention is designed to:

- Satisfy XBRL linkbase uniqueness requirements.
- Enable deterministic generation of both tuple-based and OIM-based taxonomies.
- Support structural extension through class specialization.
- Preserve semantic clarity across disparate modules and reporting domains.

5.5 Semantic Path

5.5.1 Purpose and Reference

The Semantic Path is a deterministic string used to uniquely identify and locate a semantic element within the Logical Hierarchical Model (LHM). It MUST follow the syntax rules defined in the JSONPath specification to represent the hierarchical structure and relationships between classes and properties. This path serves as the authoritative "Common Anchor" for all syntax bindings, ensuring that semantic content remains consistent across different serializations.

5.5.2 Syntax Rules and Construction

The Semantic Path MUST be constructed based on the deterministic results of the graph-walk process using the following JSONPath-compliant rules:

- Root Member (`\$`): Every path MUST begin with the symbol `\$` to represent the root entry point of the hierarchical message.
- Child Operator (`.`): Each level of the hierarchy is accessed using the dot-notation child operator.
- Segment Identification: Each segment MUST consist of the module prefix and the class or property term (format: `module:Term`).
- Node Representation:

Class Nodes: Represented as members of the hierarchy (e.g., `\$cor:AccountingEntries.cor:EntryHeader`).

Attribute Nodes: Represented as the terminal property of a class path (e.g., `\$cor:AccountingEntries.cor:EntryHeader.cor:EntryNumber`).
- Specialization Traceability: The path MUST reflect the specific LIFO (Last-In-First-Out) stack of classes active during the graph walk to ensure that specialized subclasses are uniquely identified.

5.5.3 Functional Role in the Framework

The Semantic Path acts as the primary logical key for the following taxonomy operations:

- Normalization Input: The segments within the Semantic Path provide the raw string data for the sequence-preserved deduplication and LC3 (Lower Camel Case Concatenation) normalization process required to generate XML element names.
- Binding Source: The path provides the deterministic source used to generate XPath (for XML) and JSONPath (for OIM) expressions in the Syntax Binding table.
- Reference Handling: For reference associations, the path terminates at the unique identifier (Foreign Key) of the associated class, preventing infinite recursive loops while maintaining relational integrity.

5.5.4 Conformance and Validation

Conformance MUST be verified by ensuring that:

- The Semantic Path accurately preserves the exact sequence of composition and aggregation associations traversed during the graph walk.
- The path is recorded in the LHM and retained unchanged in each derived HMD as the traceability key for taxonomy assembly.
- Any deterministic transformation between serializations (e.g., from XBRL XML to xBRL-CSV) utilizes the Semantic Path to guarantee that no semantic loss occurs.

6 Extension Mechanisms and Governance

6.1 Semantic authority and extension point

Extensions MUST be defined first in the FSM. An extension MUST NOT be introduced only in the BSM, LHM, HMD, XSD, linkbase, xBRL-CSV metadata, or instance. Those artefacts are derived views or syntax bindings and MUST preserve traceability to the FSM definition.

An extension MAY add a Class or property, specialise an existing Class or property, change an Association kind, restrict a value domain or multiplicity, prohibit an inherited property with `0` or `0..0`, or declare use or extension of a code list.

6.2 Shared, Aligned, and Distinct

Shared, Aligned, and Distinct classify semantic dependency and reuse scope, not merely folder placement or origin.

- Shared definitions are suitable for broad reuse across countries, institutions, industries, and implementations.
- Aligned definitions specialise Shared definitions while preserving dependence on a stated public business context, process, document structure, law, regulation, national or regional standard, or published industry standard.
- Distinct definitions are specific to an enterprise, corporate group, product, bilateral trading relationship, or closed community.

A definition **MUST NOT** be classified as Shared solely because it originates in CCL or another international standard. The definition, datatype, constraints, business context, and reuse scope **MUST** be assessed.

6.3 Namespace, ownership, and versioning

Each Shared, Aligned, or Distinct module **MUST** use a namespace under the authority of its maintainer. A specialised concept **MUST** retain a machine-readable relationship to its base concept but **MUST NOT** copy that concept into a new namespace merely for assembly convenience. Versions, status, deprecation, and replacement relationships **MUST** be explicit.

6.4 Extension property identity

The identity of an Attribute is ``property_term``. The identity of an Association is the ordered combination of ``association_role``, ``associated_module``, and ``associated_class``. ``association_role`` is stored in its own column immediately after ``property_term``; it **MAY** be empty. ``property_term``, Association kind, representation term, and multiplicity are not part of Association identity. A child Class **MAY** therefore change those non-identity characteristics without creating a second Association.

6.5 Extension result manifest

The framework **SHOULD** produce ``extension-manifest.json`` as a machine-readable record of an extension result. The manifest records what an Aligned or Distinct module adds to, specialises from, restricts in, or removes from Shared or another governed module. It also records code-list dependencies and alignment sources. It is a result and traceability record, not a semantic source and not an XBRL taxonomy.

JSON relationship	Meaning
extends-module	An Aligned or Distinct module extends another module.
specializes-class	An extension Class specialises a base Class.
specializes-property	An extension property specialises a base property.
adds-property	A property is added by the extension.
restricts-property	Multiplicity, value domain, or another characteristic is restricted.
prohibits-property	An inherited property is removed by <code>`0`</code> or <code>`0..0`</code> .
uses-code-list	A property uses a stated code list or enumeration domain.
extends-code-list	An extension adds governed values to a base code list.
aligns-with / derived-from	The semantic definition is aligned with or derived from an external source.
replaced-by	A later version supersedes the component.

The following structure is an illustrative minimum. The final JSON Schema and controlled relationship values remain subject to PoC validation.

```

{
  "schemaVersion": "0.1",
  "extensionId": "jp-tax",
  "classification": "aligned",
  "version": "2026-12-31",
  "status": "draft",
  "extensionModule": {"id": "jp-tax", "namespace": "..."},
  "baseModules": [{"id": "taf", "version": "2026-12-31"}],
  "relationships": [
    {"kind": "specializes-class",
     "extension": "jp-tax:JapaneseConsumptionTax",
     "base": "taf:Tax"}
  ],
  "codeLists": [
    {"property": "jp-tax:TaxCategoryCode",
     "listId": "JapaneseConsumptionTaxCategory",
     "relationship": "uses-code-list"}
  ],
  "sources": [],
  "generatedFrom": {"fsm": [], "bsm": ""}
}

```

Each recorded relationship SHOULD identify the extension component, base component, relationship kind, module and namespace, version, status, source or rationale, and the FSM row or stable semantic identifier from which it was derived. A code-list record SHOULD identify the property, list identifier and version, usage or extension relationship, authority, and syntax-binding method where known.

A dictionary taxonomy for these relationships is outside the current PoC scope. A future revision MAY define one after the JSON contract and governance requirements have been validated. Until then, processors MUST NOT require such a taxonomy.

6.6 Aligned-pool governance

Aligned extensions intended for reuse MUST be registered with their origin, definition, base relationship, datatype, multiplicity or value-domain change, code-list dependency, status, version history, change rationale, and licensing or sharing status. Part 4 defines the review and lifecycle procedures.

6.7 Build manifest and its boundary

`build-manifest.json` is a generation record, not a profile definition and not a semantic source. It SHOULD record the FSM inputs and SHA-256 values, root Class QNames and order, tool names and versions, command-line options, generation time, output checksums, validation tool versions, validation results, and applicable taxonomy entry points.

The build manifest MUST NOT select, add, remove, or redefine a Class or property independently of the FSM. If a recorded root or input conflicts with the FSM or generated artefacts, the build MUST fail or be reported as non-conformant.

`extension-manifest.json` and `build-manifest.json` have different purposes. The former records the semantic effect and provenance of an extension; the latter records how a specific set of outputs was generated and validated. They MAY reference each other by identifier and checksum but MUST remain independently interpretable.

7 Graph walk

7.1 Scope and processing stages

This Clause specifies the deterministic semantic processors represented in the current PoC by `specialization.py` and `graphwalk.py`. The normative rules apply to any implementation; the program names and command examples are informative.

Processing consists of two independent stages: (1) specialisation of one or more FSM CSV inputs into one BSM CSV, and (2) Graph Walk of that BSM from explicitly supplied root Class QNames into an LHM or root-specific HMD CSV. Neither stage requires profile data.

7.2 Specialisation: FSM to BSM

7.2.1 Inputs and registration

The processor **MUST** read one or more FSM CSV inputs in the declared order. It **MUST** normalise surrounding and repeated whitespace without changing semantic tokens. A Class row **MUST** be registered before property rows owned by that Class. A later input **MAY** reopen an existing Class to add directives, but a conflicting non-empty Class characteristic **MUST** be reported.

Recognised Class row types are ``Class``, ``Abstract Class``, and ``Super Class``. Recognised property row types are ``Attribute``, ``Composition``, ``Aggregation``, and ``Reference Association``. A ``Specialization`` row identifies a superclass through ``associated_class``.

7.2.2 QName and Class resolution

A qualified Class name **MUST** be parsed into prefix and local name at the first colon. An implementation **MUST NOT** infer a namespace from the first three characters of a Class name. A qualified name with an unknown prefix **MUST** be an input error.

An unqualified Class reference **MAY** be resolved only when its local name matches exactly one registered Class. No match or multiple matches **MUST** be an input error. Substring matching, prefix truncation, and first-candidate selection are non-conformant.

7.2.3 Inheritance resolution

Before resolving a Class, the processor **MUST** recursively resolve each declared superclass in declaration order. An undefined superclass **MUST** be an error. A Class encountered again on the active resolution stack constitutes a specialisation cycle and **MUST** be an error that reports the cycle path.

Inherited properties **MUST** be copied to the child Class before the child's direct property directives are applied. With multiple superclasses, later inherited properties having the same normative identity replace earlier inherited properties; the implementation **SHOULD** report such a collision unless the precedence is explicitly intended.

7.2.3.1 Active resolution stack

Specialisation **MUST** maintain an active LIFO resolution stack separate from the cache of Classes already resolved. A stack entry identifies the Class QName currently being resolved and its position in the superclass chain. **PUSH** occurs immediately before resolving that Class's superclasses; **POP** occurs only after all inherited properties and direct directives have been resolved and the effective property set has been cached.

Finding a Class in the completed-resolution cache means that a copy of its effective properties may be reused. Finding the same Class QName in the active LIFO stack instead means that the inheritance chain is cyclic. The processor **MUST** stop and report the complete stack path, including the repeated Class. A failed resolution **MUST NOT** leave a partially resolved Class in the cache.

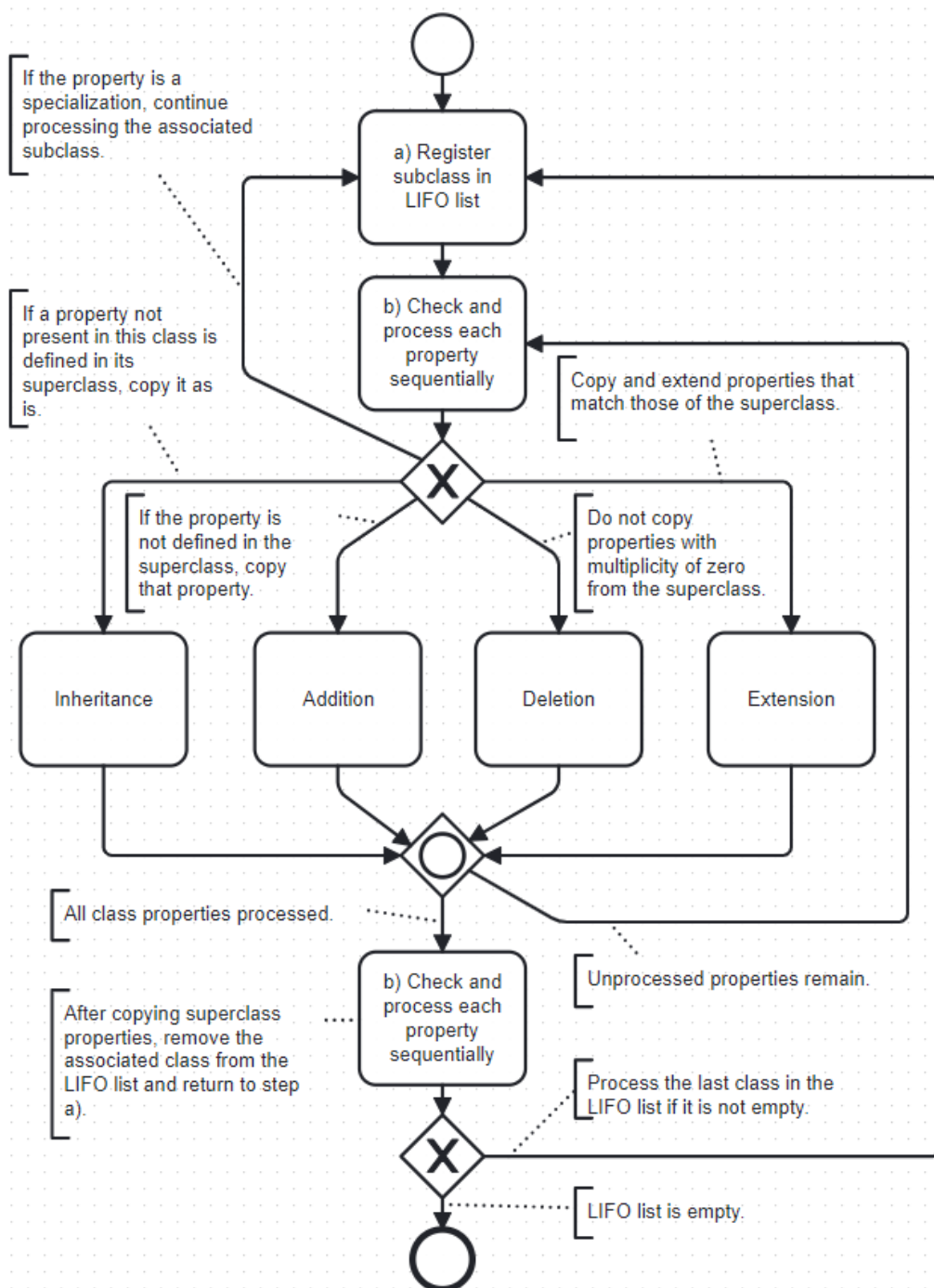


Figure 2 — Specialisation flow

Figure 2 shows the normative algorithm. Its registration in the LIFO list, sequential property processing, inheritance/addition/deletion/extension branches, return to the property loop, removal of the associated Class from the LIFO list, and empty-list termination MUST be retained. The following steps and validation rules explain the existing nodes and branches; they do not add, remove, replace, or reorder any processing path in Figure 2.

7.2.3.2 Detailed correspondence to the original flow

Step	LIFO/cache action	Required processing
S1	No stack change	Receive a Class reference and resolve it to one registered Class QName.
S2	Read cache	If already completed, return a defensive copy of the cached effective property set.

Step	LIFO/cache action	Required processing
S3	Inspect active stack	If the QName is already active, report the cycle path and terminate.
S4	PUSH Class	Create an empty ordered effective-property map for the Class.
S5	Nested PUSH/POP for each parent	Resolve each superclass recursively in declaration order; copy its properties to the child and rebuild child-owned traceability identifiers.
S6	Current Class remains on top	Merge multiple parent property sets by normative identity and declared precedence.
S7	Current Class remains on top	For each direct directive in source order: compute identity; delete for multiplicity 0 or 0..0; otherwise override a match using non-empty fields or append a new property.
S8	Current Class remains on top	Resolve every effective ('associated_module', 'associated_class') reference; reject undefined targets and exclude Associations from a concrete Class to an Abstract Class.
S9	POP then write cache	Cache a complete defensive copy only after successful resolution; later emit it only when the owning Class is concrete.

An Association's 'associated_class' is not recursively walked during Specialisation merely because it is an Association. In this stage it participates in Association identity, target validation, and output qualification. Recursive resolution in Specialisation follows only declared superclass links. Traversal through Composition, Aggregation, and Reference Associations belongs exclusively to Graph Walk.

7.2.4 Property identity

For an Attribute, identity is 'property_term'; it MUST be non-empty. For an Association, identity is ('association_role', 'associated_module', 'associated_class'). An empty 'association_role' matches only another empty role with the same 'associated_module' and 'associated_class'. Both associated-Class columns MUST be present. 'property_term' does not participate in Association identity.

'property_type', 'representation_term', and 'multiplicity' MUST NOT participate in property identity. This permits a child Class to change Reference Association to Aggregation or Composition, change datatype or representation, or change multiplicity without creating a duplicate property.

7.2.5 Override, addition, and deletion

For a directive matching an inherited property, every non-empty child field replaces the inherited value; every empty child field retains the inherited value. A non-deletion directive with no inherited match adds a new property.

Multiplicity '0' or '0..0' is a deletion instruction. It MUST remove the matching inherited property and MUST NOT be emitted as an effective BSM property. An unmatched deletion MUST be reported; strict processing MAY treat it as an error. Repeated identical deletion instructions MAY be collapsed after the first successful deletion.

7.2.6 Abstract Classes and invalid Associations

Abstract Class and Super Class rows provide inheritance but MUST NOT be emitted as concrete BSM Classes. When a concrete Class has an effective Association to an Abstract Class, the processor MUST report the Association and exclude it from the BSM. An undefined associated Class MUST be an error.

7.2.7 BSM output contract

The current PoC BSM CSV has 16 columns in this order: 'sequence', 'level', 'property_type', 'identifier', 'module', 'class_term', 'property_term', 'association_role', 'representation_term', 'associated_module', 'associated_class', 'multiplicity', 'definition', 'label_local', 'definition_local', and 'id'.

Concrete Class rows MUST have level 1 and effective property rows MUST have level 2. A logical Class is identified by (`module`, `class_term`), and a logical associated-Class reference is identified by (`associated_module`, `associated_class`). Local labels and definitions MUST be preserved. The BSM MUST NOT contain an `element` column because a BSM property may occur at more than one LHM path.

7.2.8 Specialisation conformance

The output MUST contain no Specialization rows, no deletion directives, no Abstract or Super Class rows, no unresolved Class references, no specialisation cycles, and no duplicate effective properties within a Class under the identity rules of 7.2.4.

7.3 Graph Walk: BSM to LHM

7.3.1 Inputs and root selection

Graph Walk MUST read a conformant BSM and one or more root Class QNames supplied explicitly by the caller. Each root MUST resolve to exactly one concrete BSM Class. A missing or ambiguous root MUST be an error. Root order is significant and MUST be preserved.

Supplying a root controls the hierarchy to be emitted; it does not select or change BSM semantics. Multiple root Classes MAY produce a combined LHM or separate HMD outputs, provided the chosen behaviour and ordering are recorded.

7.3.2 Class map and property order

The processor MUST build a Class map keyed by resolved Class QName. Properties MUST retain their stable BSM order. Duplicate Class keys or duplicate properties under the identity rules MUST be errors rather than silently replacing an arbitrary record.

7.3.3 Depth-first traversal

Traversal MUST be depth-first and use an explicit active-path stack. On visiting a Class occurrence, the processor MUST emit the Class, then its Attributes, then its navigable Associations in the stable order defined below. When an Association is fully processed, traversal returns to the prior Class.

To retain the existing human-readable ordering, Associations SHOULD be grouped as mandatory singular (`1` or `1..1`), optional singular (`0..1`), and plural (`0..*` or `1..*`), while preserving source order within each group. Other valid multiplicities MUST be handled by an explicitly documented stable rule.

7.3.3.1 LIFO traversal frame

Each active-path LIFO entry MUST be a traversal frame, not only a display name. A frame records at least: (a) the resolved Class QName; (b) the occurrence segment formed from the incoming Association role and associated Class; (c) the incoming Association kind and multiplicity; (d) the current semantic path; (e) the stable ordered Association list; and (f) the index of the next Association to process. This retained index is the return position when a child frame is popped.

The root frame has no incoming Association. For a non-root occurrence, the processor MUST first resolve the Association's `associated_class` to one Class-map QName. The Association role MUST qualify the occurrence segment when it distinguishes two uses of the same target Class. The target Class definition supplies the properties to be emitted; the incoming Association supplies the occurrence multiplicity and the edge semantics.

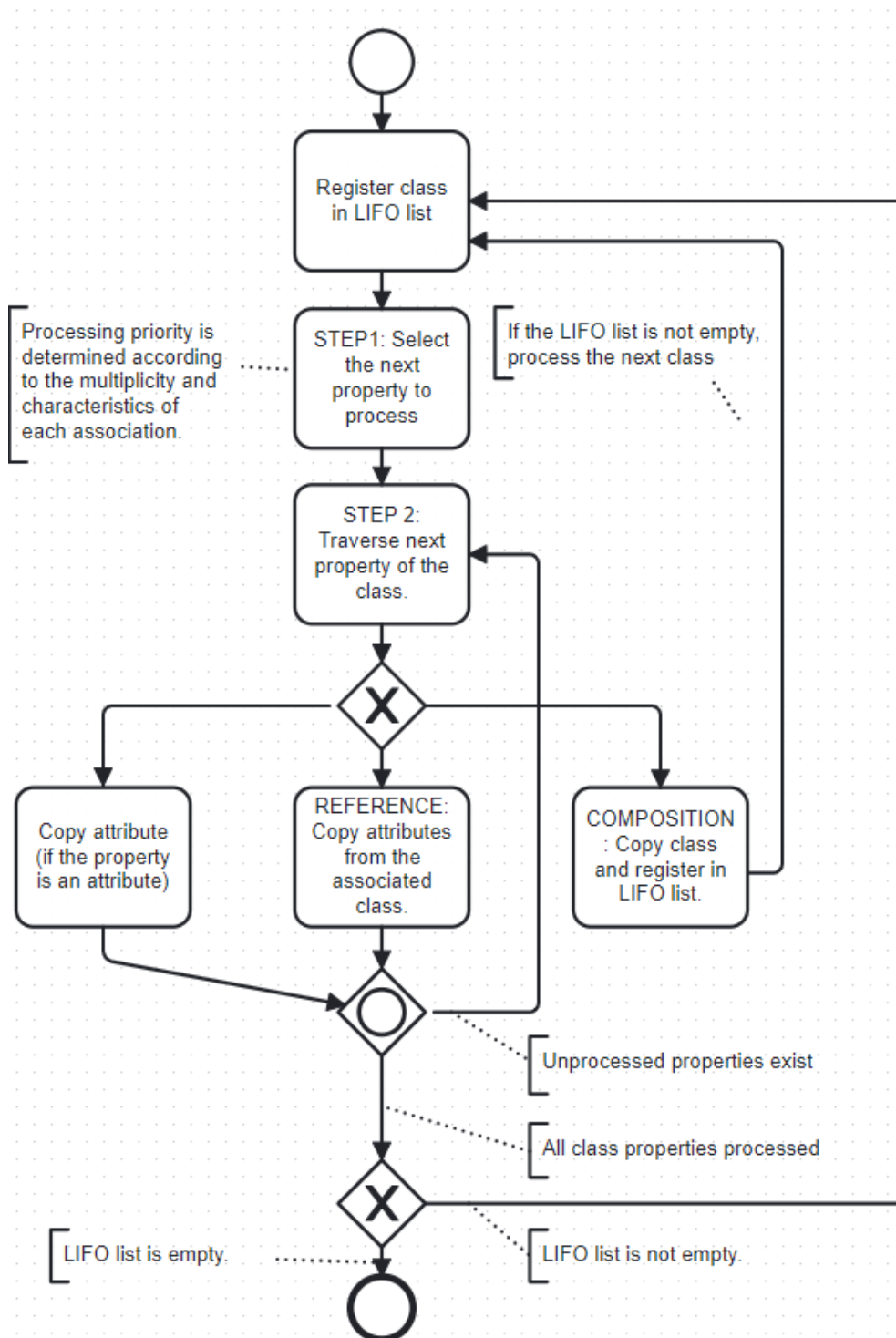


Figure 3 — Graph Walk flow

Figure 3 shows the normative algorithm. It preserves Class registration in the LIFO list, multiplicity-based property selection, traversal of the current Class property, the Attribute/Reference/Composition branches, return for unprocessed properties, and termination when the LIFO list is empty. In the current model, Aggregation uses the same navigable branch as Composition. The following steps and validation rules explain the existing nodes and branches; they do not add, remove, replace, or reorder any processing path in Figure 3.

7.3.3.2 Detailed correspondence to the original flow

Step	LIFO action	Required processing
G1	No stack change	Resolve the HMD root QName to exactly one concrete BSM Class and initialise an empty output.
G2	PUSH root frame	Set the root semantic path, output level, stable property order, and next-Association index.
G3	Read top frame	Emit the Class occurrence once, followed by all Attribute occurrences in BSM order.
G4	Advance frame index	Select the next unconsumed Association in mandatory-singular, optional-singular, then plural group order, preserving source order within each group.
G5	Current frame remains on top	Resolve ('associated_module', 'associated_class'); combine 'association_role' and target Class into the occurrence segment; derive the candidate child semantic path.
G6	Optional transient reference frame	For Reference Association, emit the R occurrence. If target PK Attribute(s) exist, emit only those rows marked REF. If no target PK exists, emit nothing below R and record a non-fatal error. Do not process target non-key properties or Associations; then resume G4.
G7	Inspect active frames	For Composition or Aggregation, if the resolved target Class QName is already active, report the stopped edge or replace it with the modelled identifier reference; do not PUSH.
G8	PUSH child frame	For a navigable non-cyclic Composition or Aggregation, create the associated-Class occurrence using incoming multiplicity and continue at G3.
G9	POP completed frame	When no Association remains, pop the frame and resume the parent at its saved next-Association index.
G10	Stack becomes empty	Finish the root-specific hierarchy, validate path/element uniqueness, and write the LHM/HMD rows.

7.3.4 Association semantics

Composition and Aggregation Associations **MUST** expand the associated concrete Class and its reachable properties. The Association's multiplicity applies to the associated Class occurrence in the resulting hierarchy.

A Reference Association **MUST** first emit the reference occurrence. When the target Class has one or more Attributes marked 'identifier=PK', Graph Walk **MUST** emit only those Attributes below the reference occurrence and **MUST** mark the copied identifiers as 'REF'. The target's non-key Attributes and Associations **MUST NOT** be expanded. When the target Class has no Attribute marked 'identifier=PK', Graph Walk **MUST** emit no row below the reference occurrence, **MUST** record a non-fatal error diagnostic identifying the Association and target Class, and **MUST** resume processing with the next property of the originating Class.

For each Association, 'association_role' is the role and the ordered pair ('associated_module', 'associated_class') is the target Class reference. The processor **MUST NOT** treat role text or 'property_term' as the Class lookup key. It **MUST** resolve the two associated-Class columns first, then use 'association_role' only to name and distinguish the hierarchical occurrence. An empty role remains valid when the target Class alone provides a unique occurrence identity.

7.3.5 Cycles and repeated occurrences

An Association that would revisit a Class already on the active path **MUST** not be expanded recursively. The processor **MUST** report the stopped edge or represent it as an identifier-based reference according to the model. This rule prevents infinite recursion.

The same Class **MAY** occur more than once through different Association paths. Such occurrences are semantically distinct hierarchical occurrences and **MUST** be distinguished by ``semantic_path``; the underlying BSM ``id`` remains unchanged for traceability.

7.3.6 Semantic path

Every emitted occurrence **MUST** have a deterministic ``semantic_path`` beginning with ``$``. Each subsequent segment **MUST** identify the module and Class or property term reached at that step. The path **MUST** preserve Association roles where they distinguish repeated uses of the same associated Class.

Semantic paths **MUST** be unique within an output. A reference path terminates at the copied reference identifier. A path **MUST NOT** depend on a filesystem location, execution time, hash-map iteration order, or a generated taxonomy element name.

7.3.7 Element-name derivation

``element`` **MUST** be derived only after ``semantic_path`` is known. The initial candidate is the module prefix plus the LC3 form of the terminal Class or property term. If that QName is already assigned to another semantic path, the processor **MUST** prepend the nearest necessary parent term or terms until the QName is unique. Duplicate words **MUST** be removed case-insensitively while preserving first occurrence and word order.

The algorithm **MUST** add only the minimum parent context required for uniqueness. If no unique QName can be produced deterministically, processing **MUST** fail rather than append an unstable counter.

7.3.8 LHM output contract

The current PoC LHM CSV has 17 columns in this order: ``sequence``, ``module``, ``level``, ``type``, ``identifier``, ``name``, ``datatype``, ``multiplicity``, ``domain_name``, ``definition``, ``label_local``, ``definition_local``, ``element``, ``id``, ``semantic_path``, ``associated_module``, and ``class_term``. ``path``, ``abbreviation_path``, ``xpath``, and ``associated_class`` are not LHM columns; physical syntax paths belong to syntax binding.

Class, Attribute, and Reference occurrences **MUST** be distinguishable by ``type``. Sequence numbers **MUST** reflect output order. Labels and definitions **MUST** remain traceable to the BSM. ``xpath`` and other serialisation paths are binding artefacts and **MAY** be generated later, but their presence **MUST NOT** alter ``semantic_path``.

7.3.9 Graph Walk conformance

For the same BSM bytes, roots, root order, and tool version, Graph Walk **MUST** produce the same row order, semantic paths, element QNames, and checksums. Processing **MUST** fail on unknown prefixes, undefined or ambiguous Classes, duplicate semantic paths, duplicate element QNames assigned to different paths, or unsupported multiplicities.

7.4 HMD derivation

An HMD is the hierarchy reachable from one specified root Class in the LHM. One FSM/BSM/LHM set **MAY** therefore yield several HMDs, such as Accounting Entries, Invoice, Order, Shipment, Receipt, or Payment. HMD derivation **MUST** preserve BSM ``id``, LHM ``semantic_path``, multiplicity, and property order.

The specified root Class is the HMD root. It MUST be identified by an unambiguous Class QName defined by the FSM and resolved in the BSM. The HMD root QName, not a display label or filename, is the normative identity of the HMD starting point.

Graph Walk MUST emit the HMD root as the first Class occurrence and MUST include only the properties reachable from that root under the traversal rules. Two HMDs with different root QNames are different hierarchical definitions even if they import the same modules or reuse many of the same Classes.

7.5 Current PoC command mapping (informative)

`specialization.py --in <fsm.csv> [--in <extension.csv> ...] --out <bsm.csv>` executes the specialisation stage. `graphwalk.py <bsm.csv> <lhm-or-hmd.csv> --root <QName>` executes Graph Walk; `--root` may be repeated where supported. No `profile.json` input is required.

7.6 Minimum test set

A conformant implementation SHOULD include tests for: an empty Association role; Association-kind and multiplicity overrides; `0` and `0..0` deletion; undefined and ambiguous QName resolution; specialisation cycles; concrete-to-Abstract Association exclusion; Reference Association key projection; cycle termination; repeated Class occurrences; semantic-path uniqueness; minimum-parent element-name disambiguation; stable ordering; and byte-for-byte reproducibility.

8 Modules, Profiles, and the Palette Model

8.1 Module concept

A module is a governed semantic unit represented in the FSM and bound into one or more XBRL taxonomy modules. Module boundaries MUST express semantic ownership and reuse scope, not merely a directory layout.

A semantic module owns the definitions of its Classes and properties. A taxonomy module owns their XBRL declarations and relationships. These two module views SHOULD correspond, but they are not identical: one is semantic organisation and the other is syntax packaging.

8.2 Baseline module composition

Module	Primary role	Typical dependency
cor	Core ledger structure and common anchors	Base module
bus	Reusable party and general business semantics	cor
muc	Multicurrency amounts and exchange information	cor
taf	Tax structures and reporting semantics	cor; jurisdiction tax modules
btx	Orders, shipments, invoices, payments, and source events	cor and bus where applicable
ehm	Enhanced measurable items and traceable quantities	cor and bus where applicable
lnk	Links to evidence and end-reporting structures	cor
sta	Statistical and other non-financial reporting	cor and ehm where applicable

The table states architectural roles, not a mandatory import closure for every use case. The exact dependency graph and root Classes MUST be declared in the FSM and realised by the generated taxonomy entry point.

8.3 Module dependencies and ownership

Each module dependency **MUST** be explicit. A Class or property **MUST** retain the module and namespace of its defining semantic owner when reused. A new concept is required only when the meaning intentionally diverges.

8.4 Profile

A Profile is an identifiable, versioned use-case configuration that brings modules and root-specific structures into an operational scope. It answers: what business purpose is served; which module versions participate; which root Class or Classes derive the HMDs; which validation and code-list constraints apply; and which serialisations and entry points are provided.

The Profile is neither a fourth semantic-model layer nor a second source of semantic truth. FSM rows define the modules, Classes, properties, roots, and constraints. BSM and LHM/HMD show their resolved effect. The Profile names and packages that configuration for users and processors.

8.5 Profile realisation

Each concrete message, ledger, or report type **SHOULD** have an explicit HMD root Class, root-specific HMD, and message entry point. Examples include Journal Entry, Trial Balance, Order, Shipment, Invoice, Receipt, and Payment. The message entry point loads the module taxonomy versions, relationships, code-list bindings, and validation rules required to process that one root-specific structure.

The normative link is one HMD root QName → one root-specific HMD → one message entry point. Human-readable message names may be used for labels and package navigation, but they **MUST NOT** replace the HMD root QName as the processing identity.

A Profile **MAY** contain one or more message entry points. For example, an Accounting Ledger Profile may contain Journal Entry and Trial Balance entry points; a Business Transactions Profile may contain Order, Shipment, Invoice, Receipt, and Payment entry points. A Profile and an entry point are therefore not necessarily one-to-one.

The combination **MUST** be reproducible from the FSM, roots, and syntax-binding rules. A separate profile dataset is **OPTIONAL** and, if present for operational convenience, **MUST** contain references only and **MUST NOT** duplicate semantic selections or constraints. The taxonomy package **SHOULD** identify the Profile's message entry points. xBRL-CSV metadata or another report definition references the entry point for the applicable message or report type.

8.6 Palette model

The Palette model is the assembly method by which reusable module taxonomies are made available and combined for a Profile. A palette is analogous to a governed set of semantic colours: modules supply reusable components; the Profile selects an intended composition; and the entry point realises that composition as a DTS.

The Palette model **MUST** preserve module and namespace ownership, **MUST** avoid copying a concept merely because it is selected, and **MUST** permit Shared, Aligned, and Distinct extension modules to be combined without modifying their source modules. Palette selection is a syntax-assembly concern; semantic inclusion and constraints remain governed by the FSM.

9 Palette Taxonomy

9.1 Purpose

A palette taxonomy is a modular XBRL implementation generated from an HMD and syntax-binding rules. It enables the same authoritative semantic model to be represented through XBRL 2.1 tuple-based and OIM-oriented dimensional constructions.

The distinction is: the Palette model is the general assembly approach; a palette taxonomy module is a reusable XBRL component; a Profile is a named use-case configuration; and a Profile entry point is the physical XBRL artefact that loads the selected components.

9.2 XBRL 2.1 palette taxonomy

The XBRL 2.1 entry point imports the required concept and content-model schemas and loads the associated linkbases. Tuple containment, XML Schema occurrence constraints, and linkbase relationships MUST be derived from the same HMD identities and multiplicities used by other bindings.

9.3 OIM-oriented taxonomy

The OIM-oriented entry point implements HMD occurrences using primary items, hypercubes, dimensions, domains, members, roles, and relationships as appropriate. xBRL-CSV metadata binds CSV tables and columns to that taxonomy; metadata is a report binding, not a semantic-model layer.

9.4 Message entry points and DTS identity

A message entry point is the physical XBRL entrance for one root-specific HMD, such as Journal Entry, Trial Balance, Order, or Invoice. It MUST load the exact module versions, extension relationships, code-list taxonomies, labels, references, and validation rules required by that structure. Concepts MUST retain their defining namespaces.

The entry point MUST identify the HMD root QName and MUST bind exactly the hierarchy derived from that root. The default cardinality is therefore one HMD root, one HMD, and one message entry point. A Profile MAY group several such entry points without merging their roots.

A package MAY also expose an aggregate discovery entry point for convenience, but conformance and report validation SHOULD resolve to the message entry point corresponding to the report's root Class. An aggregate discovery entry point MUST NOT blur or alter the constraints of its component message entry points.

9.5 Extension records

The palette DTS MAY be accompanied by `extension-manifest.json`, which documents how its Aligned or Distinct modules extend Shared or other governed modules. The JSON record is package documentation and traceability metadata; it is not loaded as a taxonomy and does not change the DTS content model.

10 How to create extensions within the XBRL GL Next Taxonomy Framework

10.1 Define the semantic extension

Define or revise the extension in an FSM module under the maintainer's namespace. State whether it is Shared, Aligned, or Distinct; identify its base Class or property where applicable; and define terms, datatype, associated Class, multiplicity, definition, labels, code-list dependency, source, and rationale.

10.2 Validate and specialise

Validate Class references, QName resolution, property identity, inheritance, deletions, Abstract Class usage, and code-list references. Run specialisation to produce a BSM and confirm that no instruction rows or unresolved references remain.

10.3 Generate hierarchies

Run Graph Walk from each intended root to generate the LHM and root-specific HMDs. Confirm stable ordering, semantic-path uniqueness, element-name uniqueness, reference handling, and cycle termination.

10.4 Generate taxonomy bindings

Generate the XBRL 2.1 palette taxonomy, OIM-oriented taxonomy, xBRL-CSV metadata, labels, references, formula or validation rules, and code-list taxonomies from the adopted HMD and binding definitions. Do not edit generated base artefacts to introduce unrecorded semantics.

10.5 Record extension results

Generate or update `extension-manifest.json` from the FSM, resolved BSM differences, module metadata, and code-list references. An Aligned extension intended for reuse MUST also follow the Part 4 registration and review procedure. A Distinct extension MUST remain clearly separated by namespace and package.

10.6 Validate and record the build

Validate the DTS and representative instances, compare expected fact and path results, and create `build-manifest.json` plus a validation report. Record input and output SHA-256 values. The manifest records the build but does not define the extension.

11 Syntax Binding

11.1 Binding source

Syntax binding uses a root-specific HMD derived from the LHM. The HMD supplies the occurrence hierarchy, BSM identifiers, semantic paths, multiplicities, datatypes, labels, and module ownership required by the binding.

The authoritative semantic source remains the FSM; the BSM, LHM, and HMD are deterministic derived artefacts. A Profile Sheet or `profile.json` MUST NOT be required to restate module, Class, property, root, or constraint information already declared in the FSM.

11.2 Binding information

For each serialised occurrence, a binding MUST identify the BSM `id`, LHM/HMD `semantic\_path`, generated concept or dimension QName, syntax path or CSV column expression, multiplicity mapping, datatype mapping, selector conditions where required, and code-list implementation.

11.3 XBRL 2.1 and OIM equivalence

XBRL 2.1 tuple nesting and OIM dimensional structures MAY use different physical constructs, but they MUST bind to the same HMD occurrences and semantic identities. Transformation is deterministic only within the explicitly mapped scope; ordering or containment semantics that cannot be preserved MUST be reported.

11.4 Validation

Binding validation MUST include DTS discovery, schema and linkbase validation, representative instance validation, semantic-path-to-concept completeness, code-list conformance, and comparison of fact values, units, periods, entities, dimensions, and reference relationships across supported serialisations.

12 Reverse engineering and migration support (informative)

12.1 Purpose

This chapter defines the conceptual foundation by which the XBRL GL Next framework supports reverse engineering and migration from tuple-based XBRL GL (XBRL 2.1) to OIM-based serialisations (for example, xBRL-CSV), while preserving semantic integrity and traceability.

This chapter establishes principles and modelling assumptions.

Detailed procedural and conformance requirements for migration are specified in Part 4 of this framework.

12.2 Background: XBRL GL XML (XBRL 2.1 / XML realisation) and implicit semantics

Tuple-based XBRL GL is a syntax-oriented design that utilises XML Schema `complexType` definitions and tuple nesting to represent accounting and business data structures.

In this design:

- hierarchical relationships are expressed through tuple containment;
- multiplicities are expressed through XML Schema cardinalities;
- structural extension is achieved by extending `complexType` definitions.

Although primarily syntax-driven, XBRL GL XML (XBRL 2.1 / XML realisation) implicitly embodies a logical hierarchical model (LHM) corresponding to the `complexType` structure defined in the XML schema. Header-detail groupings, document-level constructs, and repeating line structures can therefore be interpreted as semantic structures, even though they are not explicitly defined as a semantic model.

12.3 Principle of semantic primacy

The xBRL GL framework is based on the principle that:

Semantics are authoritative; serialisations are bindings.

Accordingly:

- and OIM-based xBRL-CSV are treated as equivalent serialisations of the same semantic model;
- migration between serialisations is achieved by recovering and reusing semantic layers, rather than by direct syntax-to-syntax transformation.

12.4 Role of the Foundational Semantic Model (FSM)

To externalise and stabilise the semantics embedded in tuple-based structures, the framework defines a Foundational Semantic Model (FSM).

The FSM:

- defines reusable abstract classes (for example, Header and Detail);
- defines common properties independently of XML Schema constructs;
- provides a stable semantic vocabulary shared across domains and modules.

These abstract classes enable structural extension through specialisation, allowing concrete classes (such as `entryHeader`, `transactionHeader`, or `reportHeader`) to inherit common semantics while introducing domain-specific refinements.

12.5 Reconstruction of semantic layers

Reverse engineering is conceptually supported by reconstructing semantic layers from existing tuple-based artefacts as follows:

a) Logical Hierarchical Model (LHM)

Tuple containment and cardinality are analysed to reconstruct hierarchical Header–Detail structures and multiplicity constraints equivalent to those produced by the forward graph-walk process.

b) Business Semantic Model (BSM)

Reconstructed hierarchies are flattened into class and property definitions, and specialisation relationships are identified.

c) Alignment with the FSM

Recovered classes and properties are aligned with the FSM and aligned semantic pool, ensuring reuse and preventing duplication.

This reconstruction restores the semantic model as an explicit, syntax-independent artefact.

12.6 Regeneration and dual syntax support

Once semantic layers have been reconstructed and aligned:

- XBRL GL XML (XBRL 2.1 / XML realisation) taxonomies MAY be regenerated to ensure consistency;
- OIM-based taxonomies (for example, xBRL-CSV) MAY be generated from the same semantic source.

This enables coexistence and parallel operation of legacy and modern serialisations during transition periods.

12.7 Summary

Reverse engineering in the xBRL GL framework is not a conversion between syntaxes.

It is a semantic recovery and regeneration process that enables existing implementations to evolve toward OIM-based representations while preserving meaning, extensibility, and auditability.

Appendix A (illustrative)

Example: Extending “Sales Order” and “Purchase Order” from abstract class “Order”

A.1 Two-phase architecture

In audit data services, the focus is not on the business transaction document itself—such as an order—but on capturing the context of the transaction within an organization. This includes information about which department processed the transaction, how it was recorded in the accounting ledger, who handled registration and approval, and the accounting period and account codes used for journal entries. The same order requires different contextual information depending on whether it is viewed from the customer’s or supplier’s perspective.

The logical model for audit data services defines common business documents as abstract classes in the FSM. These abstract definitions are then specialized into subclasses in the BSM representing how the document is registered in each business system, such as Purchase Order in the customer’s system or Sales Order in the supplier’s system. This structured approach enhances versatility and extensibility within the model.

Extended subclass definitions include information relevant to audit data, such as common elements like Activities and Accounting Period, as well as business-specific details like Purchaser in the customer’s system or Sales Person in the supplier’s system. This structured representation ensures consistency while allowing system-specific adaptations.

The two-phase graph walk ensures clear traversal by processing only the association end leading to the associated class, eliminating ambiguities from bidirectional relationships. This approach introduces deterministic specialization and inheritance processing, unlike HL7, which lacks clear rules for specialization, multiplicity constraints, and association directionality.

A.1.1 Specialization Processing (Phase 1 of Graph Walk)

Specialization is resolved first to establish inheritance before processing attributes and associations. This involves:

— Inheritance Resolution:

Sales Order: Inherits attributes from Order but removes Supplier (0) and adds Sales Person.

Purchase Order: Inherits attributes from Order but removes Customer (0) and adds Purchase Clerk.

All properties from the superclass Order (Order Number, Order Date, Transaction Amount, Seller, Buyer) are copied into BSM.

— Apply Specialization Rules:

Sales Order: Removes Supplier, adds Sales Organization and Sales Person.

Purchase Order: Removes Customer, adds Purchase Organization and Purchase Clerk.

— Finalize Class Structure:

Complete structure definition before proceeding to attribute and association traversal.

A.1.2 Normal Graph Walk Processing (Phase 2 of Graph Walk)

Once specialization is complete, the graph walk proceeds by processing attributes and relationships based on the finalized structure:

The two-phase graph walk ensures specialization is handled first, resolving inheritance before processing attributes and associations. This approach allows for the correct removal and addition of properties in the Sales Order and Purchase Order audit data models.

Phase	Purpose	Key Actions
Phase 1	Specialization Processing in FSM Ensures correct inheritance resolution before normal graph walk.	Copies all properties in the superclass and removes unneeded properties (0).
Phase 2	Normal Graph Walk in BSM Processes attributes and associations after inheritance is resolved.	Ensures structured hierarchical data transformation.

Table A.1 lists the two-layer architecture, where specialized subclasses in BSM are generated through the specialization graph walk.

- In this process, abstract class of business transaction defined in FSM are first copied to BSM using the graph walk.
- Based on extension conditions:
 - Some properties may be removed.
 - Some properties may be modified, including changes in multiplicity, datatype, association type, or association role.
 - New properties may be added. Based on extension conditions, some properties may be removed, and make modification in their multiplicity, datatype, association type, or association role, and new properties may be added.

This ensures that BSM subclasses inherit and refine properties from FSM while preserving both flexibility and structural integrity.

Table A.1 — FSM/BSM two-layer architecture

Customer Specialised subclass in BSM	Business Transaction (Abstract class in FSM)	Supplier Specialised subclass in BSM
Purchase Order	Order	Sales Order
Purchase Materials Received (Goods Receipt Ledger)	Delivery Note	Sales Shipment Made (Delivery Ledger)
Purchase Invoice	Invoice	Sales Invoice
Accounts Payable Payment Made	Bank Transfer Note	Accounts Receivable Cash Received

A.2 Abstract class 'Order' and its specialization

Table A.2 lists the abstract class Order and its subclasses Purchase Order and Sales Order in FSM. The properties of Order are categorized into transaction document-oriented data, application-oriented data, and audit-purpose data.

- Transaction document-oriented data include:
 - Unique identifiers such as ID, Date, Payment Term, and Settlement Term.
 - Trading partners, Customer and Supplier.
- Audit-purpose data include:
 - Accounting Period: Specifies the relevant period for the transaction.
 - Activities: Captures processes related to the transaction.
 - Business Segment: Identifies specific operational segments, such as departments or product lines.
- Application-oriented data are subclass-specific:
 - Purchase Order includes supplier-specific data such as Purchase Department and Purchaser.
 - Sales Order includes sales-specific data such as Sales Organization and Sales Person.
- Additionally, trivial data are removed in subclass specialization:
 - Supplier is removed from Sales Order.
 - Customer is removed from Purchase Order.

Table A.2 — Order, Purchase Order, and Sales Order before specialization in FSM

Property Type	Property term	Association role	Associated class	Multiplicity		
				Super class (Order)	Purchase Order	Sales Order
Attribute	ID (PK)			1	inherit	inherit
Attribute	Date			0..1		
Reference Association		Reference	Customer	0..1		
Reference Association		Reference	Supplier	0..1		
Composition		Transaction	Amount	0..1		
Reference Association			Payment Term	0..1		
Reference Association			Settlement Method	0..1		
Composition			Activities	undefined	0..1	0..1
Reference Association			Accounting Period		0..1	0..1
Reference Association			Business Segment		0..*	0..*
Specialization			Order	undefined	1	1
Reference Association		Reference	Customer		0 ^{a)}	undefined
Reference Association		Reference	Supplier		undefined	0 ^{b)}
Reference Association		Purchase Organization	Business Segment		0..1	undefined
Reference Association		Sales Organization	Business Segment		undefined	0..1
Reference Association		Purchaser	Employee		0..1	undefined
Reference Association		Salesperson	Employee		undefined	0..1
a) Customer is removed from Purchase Order.						
b) Supplier is removed from Sales Order.						

Specialization copies all properties of the superclass to the subclass except those with a specified multiplicity of 0, indicating removal. There is no abstract class in BSM. If a member body defines an aligned extension by further specializing the specialized subclasses Purchase Order and Sales Order, specialization allows the removal, modification, and addition of properties. The resulting BSM property table is listed in **Table A.3**.

Table A.3 — Order, Purchase Order, and Sales Order after specialization in BSM

Property Type	Property term	Association role	Associated class	Multiplicity	
				Purchase Order	Sales Order
Attribute	ID (PK)			1	1
Attribute	Date			0..1	0..1
Reference Association		Reference	Customer	removed	0..1
Reference Association		Reference	Supplier	0..1	removed
Composition		Transaction	Amount	0..1	0..1
Reference Association			Payment Term	0..1	0..1
Reference Association			Settlement Method	0..1	0..1
Composition			Activities	0..1	0..1
Reference Association			Accounting Period	0..1	0..1
Reference Association			Business Segment	0..*	0..*
Specialization			Order	1	1
Reference Association		Purchase Organization	Business Segment	0..1	undefined
Reference Association		Sales Organization	Business Segment	undefined	0..1
Reference Association		Purchaser	Employee	0..1	undefined
Reference Association		Salesperson	Employee	undefined	0..1

Appendix B (illustrative)

Example of the normal navigation graph walk

B.1 Graph Walk: Defining the LHM by copying the BSM

Figure B.1 is an example object class diagram. Associations include reference associations, aggregations, and compositions.

A reference association has a multiplicity of “0..1,” and all listed attributes are optional.

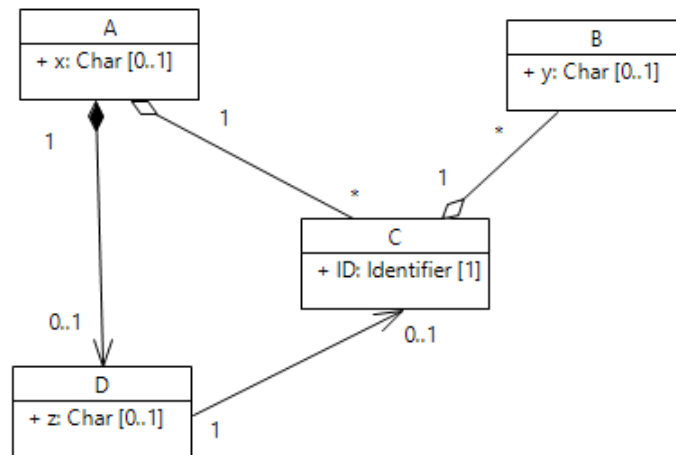


Figure B.1 — Example object class in BSM

Figure B.1 represented the object class diagram of this example.

Table B.1 — Properties of object classes in BSM

No.	Class	Property Type	Property term	Association role	Representation term	Associated class	Multiplicity
1	A	Attribute	x		String		0..1
2	A	Aggregation				C	0..*
3	A	Composition				D	0..1
4	B	Attribute	y		String		0..1
5	C	Attribute	ID		Identifier		1
6	C	Aggregation				B	0..*
7	D	Attribute	z		String		0..1
8	D	Reference Association				C	0..1

The root class is A, and the graph walk starts from there.

The graph walk navigates using the following steps:

1) Step 1: Copying Attributes

Each class’s attributes are copied completely before traversing to associated classes.

When a class is visited, all attributes are copied first before navigating any associations.

2) Step 2: Navigating Associations

After copying attributes, associations are processed by traversing one level deeper.

The associated class is placed onto the LIFO list and copied to the target model, traversing one level deeper, and steps 1 and 2 are recursively performed.

a) Cycle Prevention

If the associated class is already in the LIFO list, stop navigation to prevent cyclic traversal.

b) Reference Associations

Copy only the unique identifier of the associated class, which is the reference identifier in HMD.

The associated class is not traversed further.

c) Composition or Aggregation Associations

Add the associated class to the top of the LIFO list and copy all its properties.

Traverse the associated class recursively.

Once all properties of the class are copied and consumed, remove it from the LIFO list.

B.2 The graph walk process for the root class A

- Navigate A/ LIFO list: [A]/ Current level 2
- Step 1: Copy all attributes of class A.

No.	Level	Type	Identifier	Name	Data Type	Multiplicity
1	1	Attribute		x	String	0..1

- Step 2: Check the associated classes C and D. Based on the selection order defined by the multiplicity rule, select D first, then navigate to C. Place D on top of the LIFO list and proceed with the graph walk to D.

No.	Level	Type	Identifier	Name	Data Type	Multiplicity
1	1	Attribute		x	String	0..1
2	2	Composition		> D		0..1

- Navigate D/ LIFO list: [A, D]/ Current level 3
- Step 1: Copy all attributes of class D.

No.	Level	Type	Identifier	Name	Data Type	Multiplicity
1	1	Attribute		x	String	0..1
2	2	Composition		> D		0..1
3	3	Attribute		>> z	String	0..1

- Step 2: Check the associated classes C. Place C on top of the LIFO list and proceed with the graph walk to C.

No.	Level	Type	Identifier	Name	Data Type	Multiplicity
1	1	Attribute		x	String	0..1
2	2	Composition		> D		0..1
3	3	Attribute		>> z	String	0..1
4	3	Aggregation		>> C		0..1

- Navigate C/ LIFO list: [A, D, C]/ Current level 4
- Step 1: Copy the unique identifier of class D and complete the walk for D.

No.	Level	Type	Identifier	Name	Data Type	Multiplicity
1	1	Attribute		x	String	0..1

No.	Level	Type	Identifier	Name	Data Type	Multiplicity
2	2	Composition		> D		0..1
3	3	Attribute		>> z	String	0..1
4	3	Aggregation		>> C		0..1
5	4	Attribute		>>> ID	Identifier	1

NOTE 1 In this case, only the unique identifier is copied as a reference identifier.

- Step 2: Remove class C from the LIFO list and proceed with the graph walk to class D.
- Continue navigating D/ LIFO list: [A, D]/ Current level 3

There is no other property and finish navigation on D. Remove class D from the LIFO list and proceed with the graph walk to class A.

- Continue navigating A/ LIFO list: [A]/ Current level 2
- Step 1: No attributes remain to be copied
- Step 2: After navigating D, the remaining class is C. Check class C.

No.	Level	Type	Identifier	Name	Data Type	Multiplicity
1	1	Attribute		x	String	0..1
2	2	Composition		> D		0..1
3	3	Attribute		>> z	String	0..1
4	3	Aggregation		>> C		0..1
5	4	Attribute		>>> ID	Identifier	1
6	2	Composition		> C		0.*

- Navigate C/ LIFO list: [A, C]/ Current level 3
- Step 1: Copy all attributes of class C.

NOTE 2 If an association from A to C is an aggregation, copy all properties of C.

No.	Level	Type	Identifier	Name	Data Type	Multiplicity
1	1	Attribute		X	String	0..1
2	2	Composition		> D		0..1
3	3	Attribute		>> z	String	0..1
4	3	Aggregation		>> C		0..1
5	4	Attribute		>>> ID	Identifier	1
6	2	Composition		> C		0.*
7	3	Attribute		>> ID	Identifier	1

- Step 2: Check the associated classes B. Place B on top of the LIFO list and proceed with the graph walk to B.

No.	Level	Type	Identifier	Name	Data Type	Multiplicity
1	1	Attribute		x	String	0..1
2	2	Composition		> D		0..1
3	3	Attribute		>> z	String	0..1
4	3	Aggregation		>> C		0..1
5	4	Attribute		>>> ID	Identifier	1
6	2	Composition		> C		0.*
7	3	Attribute		>> ID	Identifier	1
8	3	Composition		>> B		0..*

- Navigate B/ LIFO list: [A, C, B]/ Current level 4

— Step 1: Copy all attributes of class B.

No.	Level	Type	Identifier	Name	Data Type	Multiplicity
1	1	Aggregation		X	String	0..1
2	2	Composition		> D		0..1
3	3	Aggregation		>> z	String	0..1
4	3	Aggregation		>> C		0..1
5	4	Attribute		>>> ID	Identifier	1
6	2	Composition		> C		0..*
7	3	Attribute		>> ID	Identifier	1
8	3	Composition		>> B		0..*
9	4	Aggregation		>>> y	String	0..1

— Step 2: Traversal of class B is complete. Remove class B from the LIFO list and proceed with the graph walk to class D.

- Continue navigating C/ LIFO list: [A, C]/ Current level 3

There is no other property and finish navigation on C. Remove class C from the LIFO list and proceed with the graph walk to class A.

- Continue navigating A/ LIFO list: [A]/ Current level 2

There are no remaining properties, completing the navigation for class A. Remove class A from the LIFO list and finalize the graph walk for class A.

The graph walk traverses an association in a deterministic depth-first search manner to reach the associated class without applying any conditional rules

Appendix C (illustrative)

Semantic Layers of the xBRL GL Framework

The XBRL GL Next Taxonomy Framework utilizes a layered semantic architecture to ensure that business data remains consistent, comparable, and traceable across multiple technical formats (such as XML, CSV, and JSON).

C.1 Foundational Semantic Model (FSM)

The FSM is the base layer of the framework. It defines jurisdiction-neutral, abstract classes, properties, and associations that represent reusable business and accounting semantics.

- **Primary Purpose:** To serve as a comprehensive, standardized catalogue of business concepts.
- **Core Components:** It includes universal concepts such as "Entity," "Account," "Amount," and "Communication".
- **Example:** In the base FSM (xBRL-GL2.0_FSM_t.csv), the Communication class is defined with standard attributes like URI Identifier, Channel Code, and Complete Number to handle contact information across all modules.

C.2 FSM Extension

An FSM Extension is used when specific business domains or jurisdictional requirements necessitate concepts not found in the international core.

- **Aligned Extensions:** Extensions intended for reuse across different jurisdictions are managed via an "aligned pool" registry to maintain global comparability.
- **Example:** The Business Transaction extension (xBRL-GL2.0_FSM_btx_t.csv) introduces the Business Transactions class. This extension adds the semantics necessary to link operational documents (like orders and shipments) to the accounting ledger.

C.3 Business Semantic Model (BSM)

The BSM is the effective semantic model generated by resolving specialisation, inheritance, additions, overrides, and deletions declared in the FSM. It is created through the Specialisation process and is not an independently authored Profile selection.

- **Refinement Process:** the Specialisation processor resolves the declared FSM Classes, constrains multiplicities, removes inherited properties whose multiplicity is `0` or `0..0`, and emits concrete Classes with their effective properties.
- **Traceability:** Each property in the BSM maintains a technical link to its originating FSM concept.
- **Example:** In the specialized BSM (xBRL-GL2.0_BSM_t.csv), the general Party class is specialized into btx:Entity Party. The attribute cor:Party_ID is inherited and assigned a specific ID (BT02_CO02_01), showing it is part of the second Business Transaction class but originates from the Core module.

C.4 Logical Hierarchical Model (LHM)

The LHM is the final logical layer, representing a flattened, hierarchical structure generated from the BSM using a deterministic Graph Walk process.

- **Hierarchical Generation:** The graph walk traverses composition and aggregation associations to expand properties into a nested tree suitable for message exchange.
- **Semantic Path:** Each element is assigned a JSONPath-compliant locator (e.g., \$.btx:BusinessTransactions.btx:Entity_Party.cor:Party_ID) which acts as the "Common Anchor" for all syntax bindings.

- Normalization (LC3): Element names are generated using Lower Camel Case Concatenation (LC3) with sequence-preserved deduplication (e.g., `btx:entityPartyId`) to ensure they are unique and concise.
- Example: The LHM for business transactions (`xBRL-GL2.0_LHM_btx_t.csv`) provides the definitive structure for developers, including hierarchical levels (1, 2, 3...), normalized element names, and XPaths (e.g., `/btx:businessTransactions/btx:entityParty/btx:entityPartyId`) for XML serialization.

C.5 HMD and HMD root

An HMD is derived for one HMD root Class QName. It retains the reachable LHM occurrences and their semantic paths for one message, ledger, or report type. Journal Entry, Trial Balance, Order, and Invoice therefore have separate HMD roots and root-specific HMDs.

C.6 Profile and message entry points

A Profile groups one or more related HMDs and message entry points for an intended use. Each message entry point implements one HMD root as a DTS. The Profile names and packages the configuration but does not redefine FSM Classes or properties.

C.7 Palette model

The Palette model supplies reusable module taxonomies and assembly rules. A Profile combines those modules through its message entry points while preserving the defining module and namespace of each concept.

C.8 Extension and build records

``extension-manifest.json`` records the semantic effect and provenance of Aligned or Distinct extensions. ``build-manifest.json`` records the inputs, tools, checksums, and validation results of one generation run. Neither JSON file is a semantic-model layer.

Appendix D (illustrative)

Semantic Layers Example: defining an extension in the FSM and generating the resulting BSM and LHM

D.1 Purpose and scope

This annex shows, using the provided example semantic-model sheets, how an extension is introduced at the Foundational Semantic Model (FSM) level, specialised into a Business Semantic Model (BSM), and transformed into a Logical Hierarchical Model (LHM) and root-specific HMD using deterministic Graph Walk.

The example is based on the layered semantic architecture used by the XBRL GL Next Taxonomy Framework to keep business data consistent, comparable, and traceable across supported serialisations.

Supported serialisations in this framework are:

- XBRL 2.1 / XML (Part 2); and
- OIM / xBRL-CSV (Part 3).

D.2 Inputs (example sheets)

The example uses the following semantic-model layer files (CSV exports from the FSM, BSM, and LHM sheets):

- Base FSM: `xBRL-GL2.0\_FSM\_t.csv` (jurisdiction-neutral abstract classes and properties; e.g., “Communication”).
- FSM extension (Business Transactions): `xBRL-GL2.0\_FSM\_btx\_t.csv` (introduces `Business Transactions` semantics linking operational documents to the ledger).
- BSM: `xBRL-GL2.0\_BSM\_t.csv` (specialised refinement of selected FSM concepts; maintains traceability to FSM).
- LHM (btx): `xBRL-GL2.0\_LHM\_btx\_t.csv` (hierarchical structure generated from BSM via graph walk; includes semantic paths, normalised names, and XPath(s)).

D.3 Design policy used in the example

- a) FSM provides a shared semantic base: reusable abstract concepts (Entity, Party/Person, Account, Amount, Communication, etc.).
- b) Aligned extensions are introduced as FSM extensions when domain/jurisdiction semantics are needed beyond the international core, with reuse governed through an “aligned pool” style mechanism to preserve comparability.
- c) BSM specialisation selects necessary elements (and may constrain multiplicities, including removal via multiplicity 0) while preserving traceability to the originating FSM concepts.
- d) LHM is derived deterministically by a depth-first graph walk over composition and aggregation, producing the complete logical hierarchy for the target domain.
- e) Semantic Path is the common anchor: every LHM element is assigned a JSONPath-compliant semantic locator used as the authoritative key for syntax binding across serialisations.

D.4 Step-by-step procedure (FSM → BSM → LHM)

D.4.1 Step 1 — Define the extension in the FSM (FSM_btx extends FSM)

Goal: introduce domain semantics (Business Transactions) without redefining shared concepts.

Procedure (example pattern):

- a) Create an FSM extension file for the module (here `btx`), which introduces at least the root class for the module's reporting domain (e.g., `Business Transactions`).
- b) Ensure the extension's intent is consistent with the overall modular architecture and independent reportability: for the btx domain, the module uses its own root element and is not anchored on the ledger root.
- c) Where cross-jurisdiction reuse is intended, manage the extension as an aligned extension (aligned pool governance) to maintain semantic equivalence and avoid duplication.

D.4.2 Step 2 — Specialise to create the BSM (select necessary elements)

Goal: create a concrete effective semantic model by resolving the required Classes and properties from the base FSM and FSM extension.

Procedure (example pattern):

- a) Resolve the declared FSM Classes and apply specialisation to create the effective BSM. The BSM is a deterministic refinement of the FSM and is not independently selected by a Profile.
- b) Constrain multiplicities and remove irrelevant properties (e.g., by setting multiplicity to 0) while preserving traceability to the originating FSM concept.
- c) Example (Party specialisation): in `xBRL-GL2.0\_BSM\_t.csv`, the general Party concept is specialised into `btx:Entity Party`. The inherited attribute `cor:Party\_ID` receives its effective BSM identifier while retaining traceability to the core semantic definition.
- d) Align this step with the "avoid semantic misuse" principle: core concepts should be defined and extended so implementers do not place person attributes into verification-metadata or "closest available" fields.

(For the driver requirement in the requirements document, see "first-class party/person identification".)

D.4.3 Step 3 — Generate the LHM via deterministic graph walk

Goal: produce the logical hierarchy and the root-specific HMD, with stable anchors for syntax bindings.

Procedure (example pattern):

- a) Perform deterministic depth-first traversal across composition/aggregation to expand the BSM into a hierarchical structure.
- b) Assign a Semantic Path to each element. The Semantic Path MUST be JSONPath-compliant and acts as the common anchor for bindings.
- c) Generate normalised element names using LC3 (Lower Camel Case Concatenation) with sequence-preserved deduplication.
- d) Record, in the LHM layer, the hierarchical levels, normalised names, and XPaths for XML serialisation.

Worked example (from the annexed description in Part 1):

- Semantic Path example: `\$..btx:BusinessTransactions.btx:Entity\_Party.cor:Party\_ID`
- Normalised element name example: `btx:entityPartyId`
- XPath example: `/btx:businessTransactions/btx:entityParty/btx:entityPartyId`

D.5 Conformance checks recommended for this workflow

When producing/maintaining these layers, the following checks are expected as baseline:

- ID uniqueness: all `id` and `element\_id` values are unique, and no accidental duplicates arise from placement-only differences.
- Semantic Path conformance: ensure the path preserves the exact sequence of associations traversed and is recorded in the LHM layer as the "master manifest" for assembly/binding.

D.6 Expected outputs and how they are used

- FSM / FSM extension: the reusable concept catalogue plus domain additions (shared core + aligned additions).
- BSM: the effective semantic model after inheritance, additions, overrides, and deletions have been resolved, with traceability to the FSM.
- LHM: the complete logical hierarchy for the target domain. For syntax binding and taxonomy assembly, an HMD is derived from a specified LHM root Class.
- Semantic-model artefact set: the FSM is the authoritative input; the BSM and LHM/HMD are generated results. Profile entry points and package metadata reference these artefacts without redefining their semantics.

Appendix E (illustrative)

Example: “VAT category taxable amount” (EN 16931 BT-116)

E.1 UBL 2.1 (Peppol BIS Billing 3.0 – UBL Invoice)

In UBL, VAT breakdown is represented by one or more `cac:TaxSubtotal` elements under `cac:TaxTotal`. The VAT category taxable amount is the `cbc:TaxableAmount` within each `cac:TaxSubtotal`, and its meaning is explicitly “sum of all taxable amounts subject to a specific VAT category code and rate”.

Typical XPath (illustrative):

```
/Invoice
/cac:TaxTotal
/cac:TaxSubtotal[
  cac:TaxCategory/cac:TaxScheme/cbc:ID = 'VAT'
  and cac:TaxCategory/cbc:ID = 'S'
  and cac:TaxCategory/cbc:Percent = 10
]
/cbc:TaxableAmount
```

Notes:

- The selector uses `TaxScheme/ID='VAT'`, plus `TaxCategory/ID` (category code) and, where present, `Percent` (rate).
- `cbc:TaxableAmount/@currencyID` MUST align with the document currency rules in the applicable profile (e.g., invoice currency).

E.2 UN/CEFACT CII D16B (EN 16931 CII)

In CII, the VAT breakdown is carried in repeated `ram:ApplicableTradeTax` elements under header settlement. The VAT category taxable amount (BT-116) is carried in `ram:BasisAmount` within each `ram:ApplicableTradeTax` (with the category and rate carried alongside it). This is shown explicitly in CII guidance where `ram:BasisAmount` is labelled as “BT-116”.

Typical XPath (illustrative):

```
/rsm:CrossIndustryInvoice
/rsm:SupplyChainTradeTransaction
/ram:ApplicableHeaderTradeSettlement
/ram:ApplicableTradeTax[
  ram:TypeCode = 'VAT'
  and ram:CategoryCode = 'S'
  and ram:RateApplicablePercent = 10
]
/ram:BasisAmount
```

Notes:

- `ram:TypeCode='VAT'` MUST be used to ensure the tax is VAT (and not another tax type) when present.
- Category and rate selection uses `ram:CategoryCode` and `ram:RateApplicablePercent`.

E.3 Example syntax binding table entry

Column	Value
Semantic group	BG-23 VAT BREAKDOWN
Semantic term	BT-116 VAT category taxable amount
Semantic path	/BG-23/BT-116

Column	Value
UBL 2.1 target (XPath)	<pre> /Invoice /cac:TaxTotal /cac:TaxSubtotal [cac:TaxCategory /cac:TaxScheme /cbc:ID = 'VAT' and cac:TaxCategory/cbc:ID = '\$vatCategoryCode' and cac:TaxCategory/cbc:Percent = \$vatRate] /cbc:TaxableAmount </pre>
UBL value / attribute	`cbc:TaxableAmount` ; currency: `@currencyID`
CII target (XPath)	<pre> /rsm:CrossIndustryInvoice /rsm:SupplyChainTradeTransaction /ram:ApplicableHeaderTradeSettlement /ram:ApplicableTradeTax [ram:TypeCode = 'VAT' and ram:CategoryCode = '\$vatCategoryCode' and ram:RateApplicablePercent = \$vatRate] /ram:BasisAmount` </pre>
CII value / attribute	`ram:BasisAmount` ; currency: `@currencyID`
Selector condition (text)	Select the VAT breakdown entry for the specified VAT category code (`\$vatCategoryCode`) and, where present, the VAT rate (\$vatRate), ensuring the scheme/type is VAT

`\$vatCategoryCode` and `\$vatRate` are parameters derived from the breakdown record (e.g., category and rate fields) or provided by the profile. XPath namespace prefixes MUST be declared in the binding specification.

E.4 Syntax binding example: XBRL GL tuple palette taxonomy

The current XBRL GL 2.1 tuple design does not provide a header-level VAT breakdown structure analogous to EN 16931 “BG-23 VAT BREAKDOWN” (i.e., per VAT category totals at the document header level). Instead, VAT data is carried at line level in `cor:entryDetail/cor:taxes`.

- Line-level taxable basis amount

The following XPath selects the taxable basis amount for a given VAT category (and, optionally, rate) at line level:

```

/xbrli:xbrl
/cor:accountingEntries
/cor:entryHeader
/cor:entryDetail
/cor:taxes[
  cor:taxTableCode = 'VAT'
  and cor:taxCode = 'S'
  and cor:taxPercentageRate = 10
]
/cor:taxBasis

```

NOTE 1: Because XBRL GL element names are unique in the taxonomy schema, the XPath MAY be shortened using a descendant selector, for example:

```
//cor:taxes[cor:taxTableCode='VAT' and cor:taxCode='S']/cor:taxBasis
```

- Header-level subtotal (computed)

Because a header-level VAT breakdown is not modelled explicitly, the subtotal for a given VAT category within an `entryHeader` is obtained by summing the matching line-level `taxBasis` values:

```

sum (
  /xbrli:xbrl
  /cor:accountingEntries
  /cor:entryHeader
  /cor:entryDetail

```

```

/cor:taxes[
  cor:taxTableCode      = 'VAT'
  and cor:taxCode        = 'S'
  and cor:taxPercentageRate = 10
]
/cor:taxBasis
)

```

NOTE 2: Because XBRL GL element names are unique in the taxonomy schema, the XPath MAY be shortened using a descendant selector, for example:

```
sum(//cor:taxes[cor:taxTableCode='VAT' and cor:taxCode='S']/cor:taxBasis)
```

NOTE 3: The shortened XPath uses the descendant selector // and therefore matches any cor:taxes element anywhere in the instance document. If the instance contains multiple cor:accountingEntries and/or cor:entryHeader records, the XPath SHOULD be scoped (for example, by restricting to a specific cor:entryHeader such as by cor:entryNumber) to avoid unintentionally selecting taxes from other entries.

Appendix F

References

(Reference list under collaborative review. Official XBRL specifications are available at <https://specifications.xbrl.org/>.)

- RFC2119 The Internet Engineering Task Force (IETF). "Key words for use in RFCs to Indicate Requirement Levels, March 1997"
Scott Bradner.
(See <http://www.ietf.org/rfc/rfc2119.txt>)
- SCHEMA-1 World Wide Web Consortium (W3C). "XML Schema Part 1: Structures"
(See <http://www.w3.org/TR/xmlschema-1/>)
- TRP XBRL International Inc. (XII). "XBRL Taxonomy Recognition Process, 28 October 2004"
Peter Calvert, and Joseph MacDonald.
(See <https://xbrl.org/taxonomyrecognition/taxonomy-recognition-acknowledgement-process-2013-04-09.htm>)
- XBRL XBRL International Inc. (XII). "Extensible Business Reporting Language (XBRL) 2.1 Specification with corrected errata dated 2 July 2008"
Phillip Engel, Walter Hamscher, Geoff Shuetrim, David vun Kannon, and Hugh Wallis.
(See <https://specifications.xbrl.org/>)
- XLINK World Wide Web Consortium (W3C). "XML Linking Language (XLink) Version 1.0."
Steve DeRose, Eve Maler, and David Orchard.
(See <http://www.w3.org/TR/xlink/>)

Appendix G (illustrative)

Intellectual property status

Copyright and license terms are under review. This draft is shared only with invited project participants through the private project repository. No permission for public redistribution or publication is granted by this draft notice.

References to external specifications and other third-party materials remain subject to the terms of their respective rights holders.

This draft does not claim approval, endorsement, publication, or official status by XBRL Japan, XBRL Europe, XBRL International, or any other organization.

Appendix H (illustrative)

Document history

Date	Author	Details
6 January 2026	SAMBUICHI, Nobuyuki	v0.1 Initial collaborative review draft.
28 July 2026	SAMBUICHI, Nobuyuki	Clarified FSM authority; detailed Specialisation and Graph Walk; defined HMD roots, Profiles, message entry points, module composition, and the Palette model; introduced extension- and build-manifest JSON responsibilities.
29 July 2026	SAMBUICHI, Nobuyuki	Aligned FSM/BSM column contracts and Association identity with FSM_q.xlsx; separated association_role from property_term; retained associated_module and associated_class; removed BSM element; confirmed the 17-column LHM contract; specified non-fatal diagnostics and continuation when a Reference Association target has no PK.

Appendix I (illustrative)

Errata corrections in this document

This appendix records editorial corrections incorporated into this collaborative review draft. The entries have not been approved or endorsed by XBRL Japan, XBRL Europe, XBRL International, or any other organization.

No errata have been incorporated into this document.